Computer Vision
for Construction Safety

Panduan langkah demi langkah membuat, melatih, dan memakai model deteksi APD dengan YOLO11, Roboflow, Streamlit, dan GitHub.



PEMATERI

Muhamad Chaerul Anwar

2026

Daftar Isi

1	Pendahuluan	1
2	Persiapan	3
3	Mengenal Dataset	6
4	Melabeli Data Sendiri	8
5	Jalur A: Training Cepat dengan Roboflow	15
6	Jalur B: Training Model Sendiri di Google Colab	23
7	Menjalankan dan Menguji Web Deteksi	30
8	Memahami Kode Web Deteksi	36
9	Menyimpan Proyek ke GitHub	43
10	Bonus: Membuat Web Bisa Dibuka dari Mana Saja	50
11	Kalau Ada Masalah	52

1. Pendahuluan

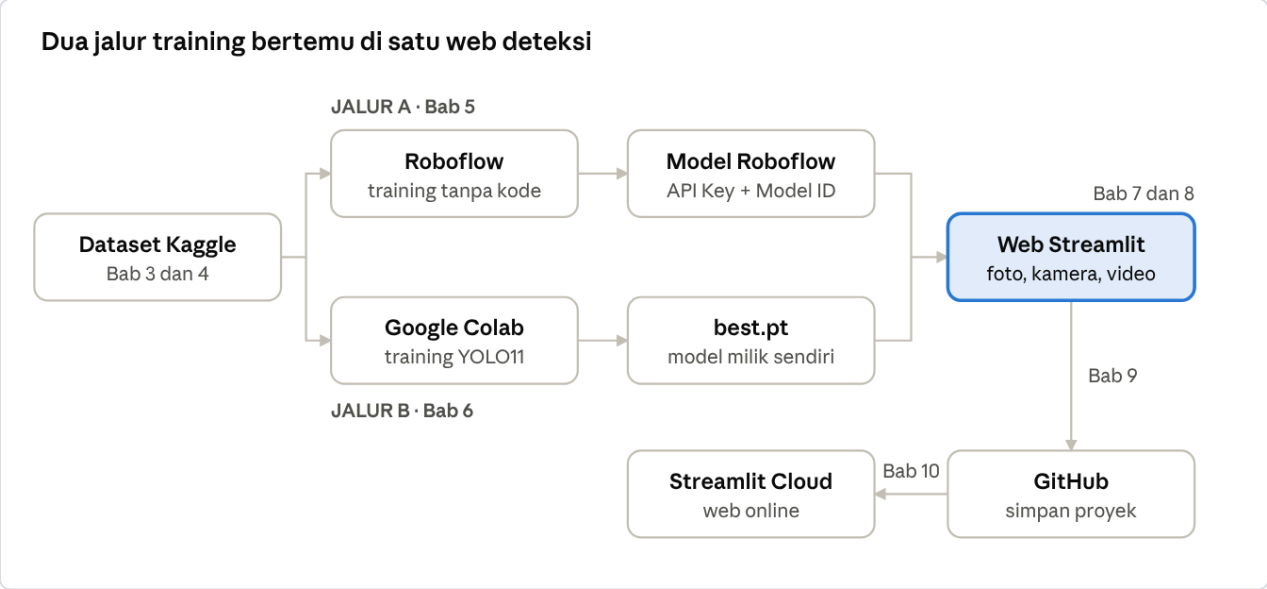
Selama workshop ini Anda membuat web sederhana yang bisa mengenali pekerja tanpa helm, rompi, atau masker. Gambarnya bisa dari foto, kamera laptop, atau video. Di akhir sesi, proyek itu Anda simpan di GitHub.

Teknik yang kita pakai namanya object detection. Model membaca gambar, menggambar kotak di sekitar objek yang dikenalinya, lalu memberi nama kelasnya, misalnya Hardhat atau NO-Safety Vest. Petugas K3 bisa memakai cara ini untuk memeriksa foto lapangan atau rekaman CCTV tanpa melihatnya satu per satu.

Dua cara melatih model

	Jalur A: Roboflow	Jalur B: Training sendiri
Cara kerja	Klik di website Roboflow, tanpa menulis kode	Menjalankan notebook Python di Google Colab
Lama training	15 sampai 60 menit, tergantung antrean	Sekitar 40 menit
Hasil	Model tersimpan di server Roboflow, dipakai lewat API	File `best.pt` yang Anda simpan sendiri
Cocok untuk	Demo cepat	Memahami proses training, bisa dipakai tanpa internet

Model dari jalur mana pun bisa dipakai di web yang sama. Gambar di bawah merangkum urutan bab di buku ini.



Gambar 1.1 Alur workshop: dua jalur training bertemu di satu web deteksi

Isi folder proyek

Semua kode untuk workshop ini ada di satu folder di github.com/est57/deteksi-apd. Isinya seperti ini:

File / folder	Kegunaan
`app/app.py`	Web deteksi
`notebooks/training_yolo_colab.ipynb`	Notebook training untuk Google Colab
`download_data.py`	Mengunduh dataset dari Kaggle
`train.py`	Training di laptop (opsional)
`data/data.yaml`	Lokasi gambar dan daftar kelas
`models/best.pt`	Model hasil training
`requirements.txt`	Daftar library Python

2. Persiapan

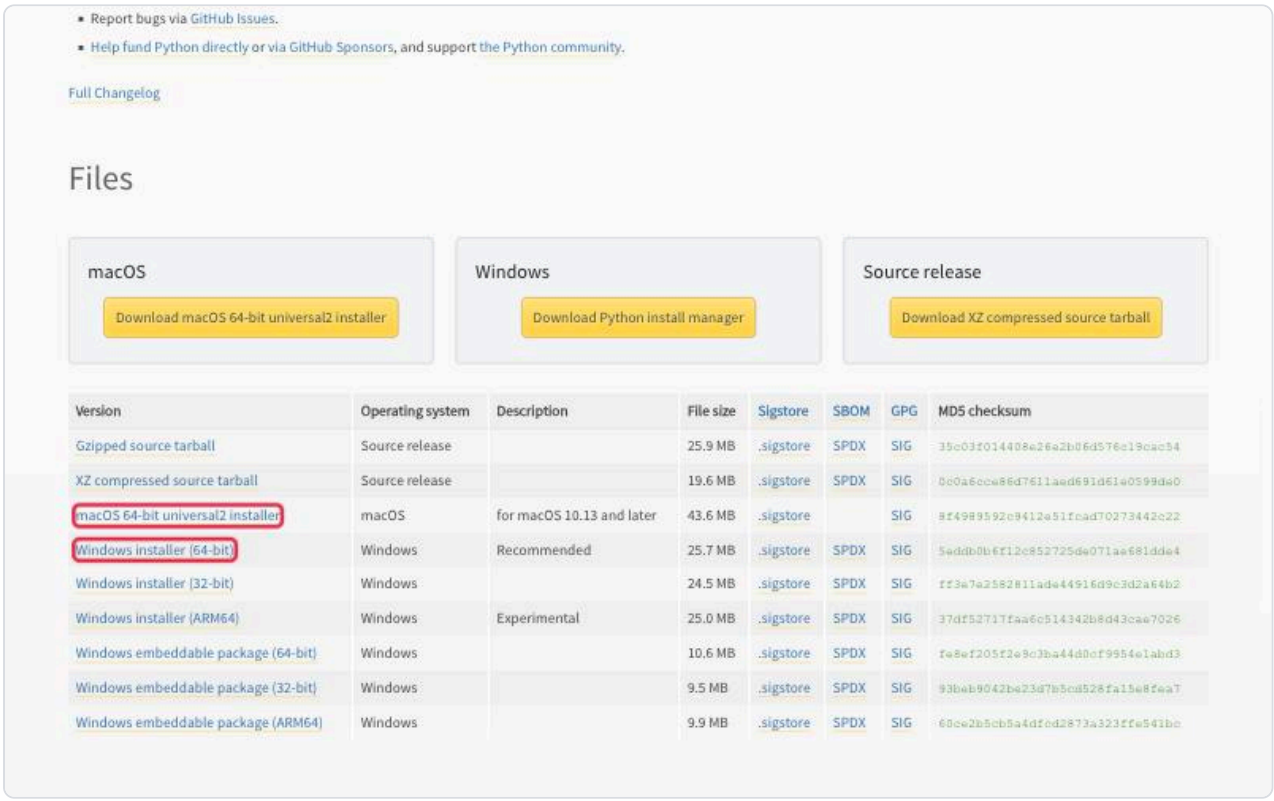
Semua akun dan aplikasi di bab ini gratis. Siapkan sebelum hari workshop supaya waktu di kelas bisa dipakai untuk praktik.

Akun

- ☐ Kaggle, tempat dataset
- ☐ Roboflow, untuk Jalur A
- ☐ GitHub, untuk menyimpan proyek
- ☐ Akun Google, untuk membuka Google Colab

Langkah 1: Install Python 3.12

1. Buka halaman Python 3.12.10 dan gulir ke bagian **Files**.
2. Pengguna Windows mengunduh **Windows installer (64-bit)**. Pengguna Mac mengunduh **macOS 64-bit universal2 installer**.



Gambar 2.1 Halaman unduhan Python 3.12.10, dua file installer ditandai kotak merah

3. Jalankan file yang terunduh. Di Windows, centang **Add python.exe to PATH** di layar pertama sebelum menekan **Install Now**. Kalau lupa, perintah `python` tidak akan dikenali nanti.

Langkah 2: Install Git dan VS Code

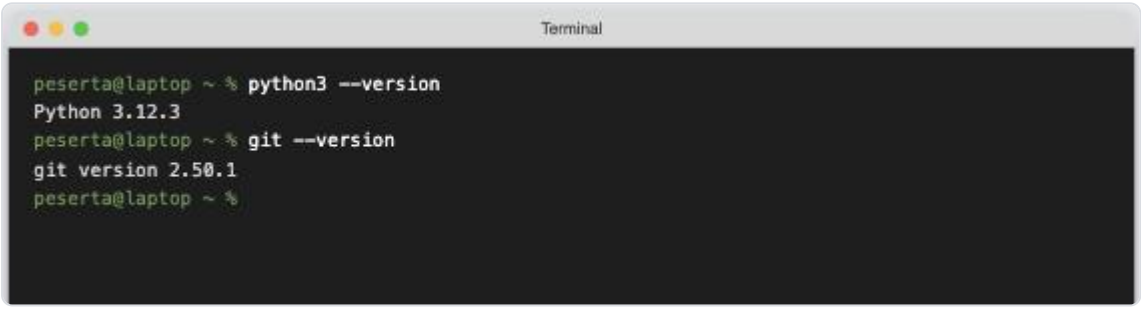
1. Unduh Git dari git-scm.com/downloads. Di Windows, tekan **Next** sampai selesai tanpa mengubah pilihan apa pun.
2. Unduh Visual Studio Code dari code.visualstudio.com. Kita memakainya untuk membuka dan mengedit kode.

Langkah 3: Cek hasil instalasi

Buka **Terminal** (Mac) atau **Command Prompt** (Windows). Ketik dua perintah ini satu per satu, tekan Enter setiap selesai mengetik:

```
python --version
git --version
```

Kalau muncul nomor versi seperti di gambar, instalasi berhasil. Di Mac, ganti `python` dengan `python3`.



Gambar 2.2 Terminal menampilkan Python 3.12.3 dan git version 2.50.1

Langkah 4: Siapkan folder proyek

1. Buka github.com/est57/deteksi-apd, klik tombol hijau **Code**, lalu **Download ZIP**. Ekstrak file ZIP-nya.
2. Buka VS Code, pilih **File > Open Folder**, lalu pilih folder proyek tadi.
3. Buka terminal di dalam VS Code lewat menu **Terminal > New Terminal**.
4. Buat virtual environment. Ini folder khusus tempat library proyek disimpan, supaya tidak bercampur dengan proyek lain.

```
python -m venv .venv
```

5. Aktifkan virtual environment:

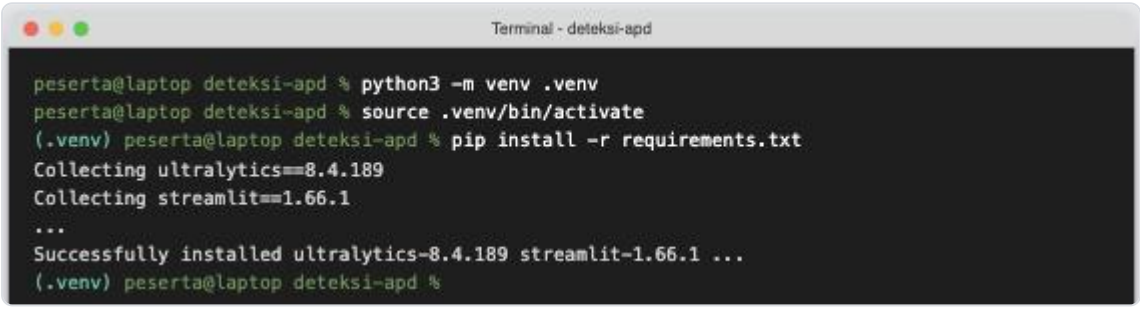
```
# Mac
source .venv/bin/activate

# Windows
.venv\Scripts\activate
```

6. Install library yang dibutuhkan. Proses ini makan waktu 3 sampai 10 menit, tergantung kecepatan internet.

```
pip install -r requirements.txt
```

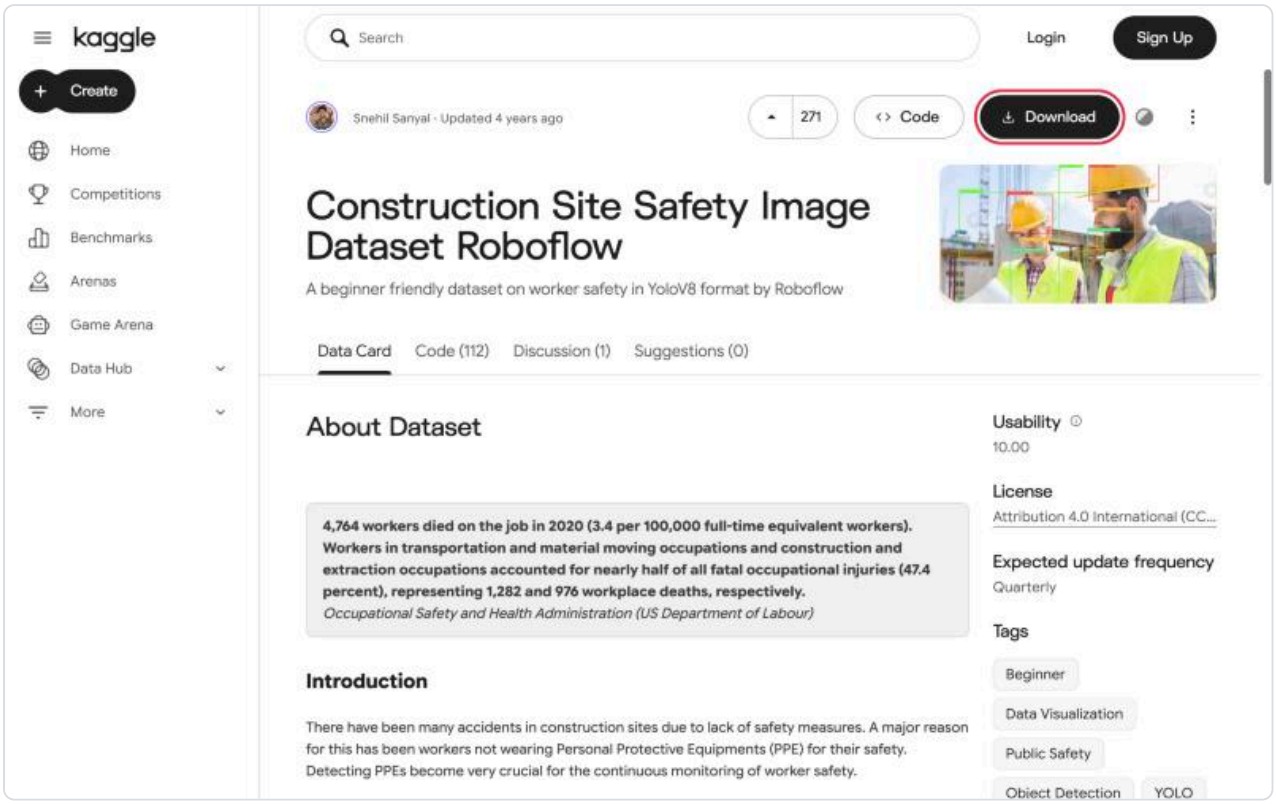
Perhatikan tulisan `(.venv)` di awal baris. Tulisan itu tanda virtual environment sedang aktif. Setiap kali membuka terminal baru, aktifkan lagi dengan perintah di langkah 5.



Gambar 2.3 Terminal setelah membuat dan mengaktifkan virtual environment, lalu menjalankan pip install

3. Mengenal Dataset

Kita memakai Construction Site Safety Image Dataset dari Kaggle. Isinya 2.801 foto area konstruksi. Setiap orang, helm, rompi, dan alat berat di foto sudah diberi kotak dan nama kelas oleh pembuat dataset.



Gambar 3.1 Halaman dataset di Kaggle, tombol Download ditandai kotak merah

Pembagian gambar

Bagian	Jumlah	Kegunaan
train	2.605	Bahan belajar model
valid	114	Dicek setiap akhir epoch untuk melihat kemajuan
test	82	Ujian akhir, gambar yang belum pernah dilihat model

10 kelas yang dikenali

No	Kelas	Arti	Jumlah kotak di train
0	Hardhat	Memakai helm	3.145

No	Kelas	Arti	Jumlah kotak di train
1	Mask	Memakai masker	1.651
2	NO-Hardhat	Tidak memakai helm	2.317
3	NO-Mask	Tidak memakai masker	3.097
4	NO-Safety Vest	Tidak memakai rompi	3.962
5	Person	Orang	9.532
6	Safety Cone	Kerucut pengaman	3.366
7	Safety Vest	Memakai rompi	3.033
8	machinery	Alat berat	5.247
9	vehicle	Kendaraan	1.545

Kelas yang diawali NO- adalah pelanggaran. Web kita nanti memberi kotak merah untuk tiga kelas itu.

Isi file label

Setiap gambar punya file teks bernama sama di folder `labels`. Satu baris mewakili satu objek:

```
5 0.512 0.430 0.210 0.650
```

Angka pertama adalah nomor kelas, di sini 5 (Person). Dua angka berikutnya posisi titik tengah kotak, dan dua angka terakhir lebar serta tinggi kotak. Keempatnya ditulis sebagai perbandingan terhadap ukuran gambar, jadi 0.512 berarti 51,2% dari lebar gambar.

Mengunduh dataset ke laptop

Langkah ini hanya perlu kalau Anda mau training di laptop. Notebook Colab mengunduh dataset sendiri.

```
python download_data.py
```

Dataset sekitar 200 MB akan tersimpan di folder `data/css-data`.

4. Melabeli Data Sendiri

Dataset Kaggle yang kita pakai sudah berlabel, jadi kita bisa langsung training. Di proyek nyata, Anda biasanya mulai dari foto lapangan yang belum punya label sama sekali. Bab ini menjelaskan cara memberi label sendiri, dari foto mentah sampai siap dilatih.

Apa itu label

Label adalah kunci jawaban yang kita berikan kepada model saat belajar. Dalam object detection, satu label terdiri dari dua hal: kotak yang mengelilingi objek dan nama kelas objek itu. Model mempelajari ribuan contoh seperti ini, lalu mencoba menebak sendiri di foto yang belum pernah dilihatnya.

Label disimpan sebagai file teks berformat YOLO, seperti yang dibahas di Bab 3. Anda tidak perlu menulis angka-angkanya sendiri. Aplikasi labeling menghitungnya otomatis setiap kali Anda menggambar kotak.

Alur dari foto mentah sampai model

1. **Kumpulkan foto** dari lokasi proyek. Usahakan bervariasi: pagi dan sore, dekat dan jauh, berbagai sudut, berbagai lokasi.
2. **Tentukan daftar kelas dan aturan labeling** sebelum mulai, supaya semua orang melabeli dengan cara yang sama.
3. **Upload foto** ke Roboflow.
4. **Gambar kotak dan pilih kelas** untuk setiap objek di setiap foto.
5. **Masukkan ke dataset** foto yang sudah selesai dilabeli.
6. **Bagi data** menjadi train, valid, dan test.
7. **Latih model** di Roboflow (Bab 5), atau ekspor ke format YOLO lalu latih di Colab (Bab 6).

Berapa banyak foto yang dibutuhkan

Tidak ada angka pasti. Sebagai titik awal, siapkan sekitar 100 sampai 300 foto untuk setiap kelas. Variasi lebih penting daripada jumlah: 200 foto dari berbagai lokasi lebih berguna daripada 1.000 foto dari satu kamera CCTV yang sama.

Cara yang praktis adalah mulai dari sedikit, latih model, lihat kelas mana yang paling lemah di confusion matrix, lalu tambah foto untuk kelas itu.

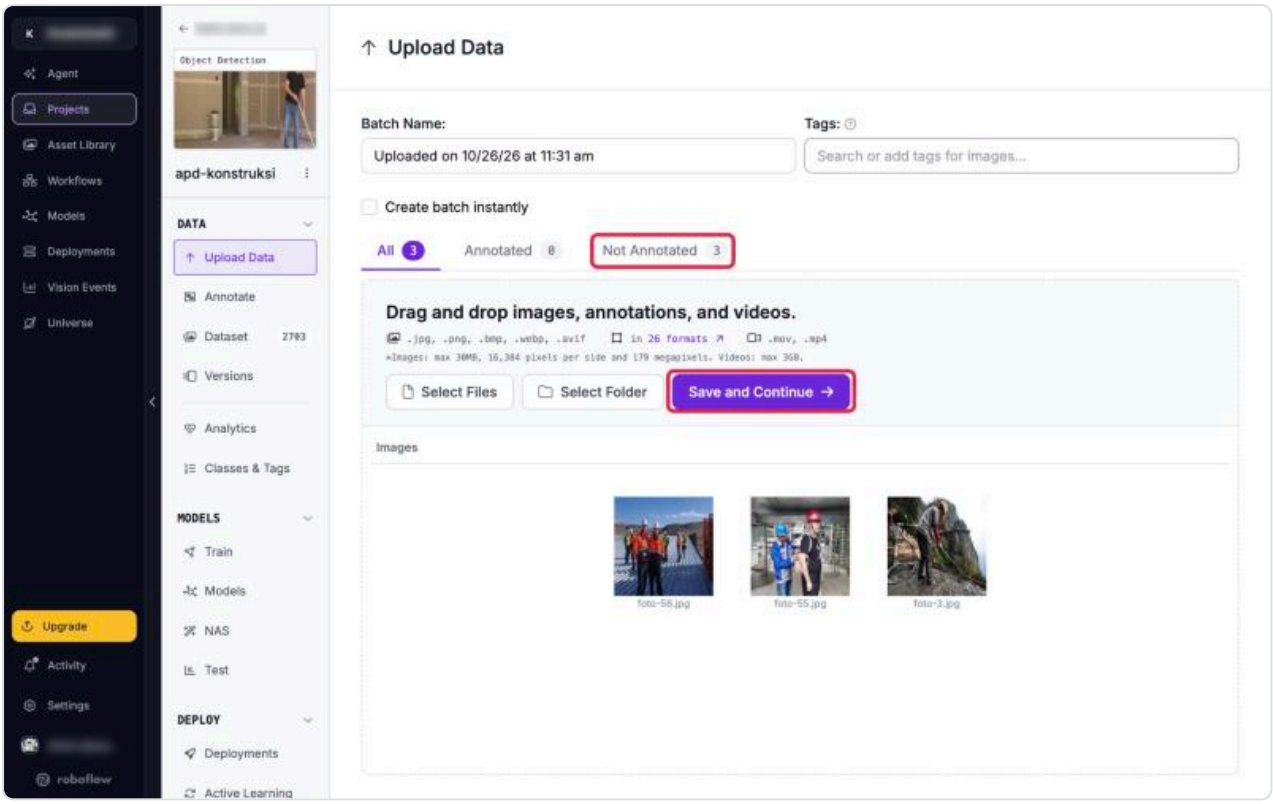
Aturan labeling

Model meniru cara Anda melabeli. Kalau labelnya tidak konsisten, model ikut bingung.

Aturan	Contoh
Kotak pas dengan objek, tidak terlalu longgar	Kotak Hardhat hanya mengelilingi helm, bukan seluruh kepala dan bahu
Labeli semua objek di foto	Pekerja kecil di kejauhan tetap dilabeli. Objek yang terlewat akan dianggap latar belakang oleh model
Objek yang tertutup sebagian tetap dilabeli	Pekerja yang setengah badannya terhalang alat berat tetap diberi kotak Person
Pelanggaran ditandai di bagian tubuh yang relevan	NO-Hardhat di kepala tanpa helm, NO-Safety Vest di badan tanpa rompi
Satu orang boleh punya beberapa label	Satu pekerja bisa mendapat Person, Hardhat, dan NO-Safety Vest sekaligus
Kasus yang meragukan disepakati bersama	Catat contoh kasus sulit, misalnya topi biasa yang mirip helm, di dokumen aturan tim

Praktik langkah 1: Upload foto tanpa label

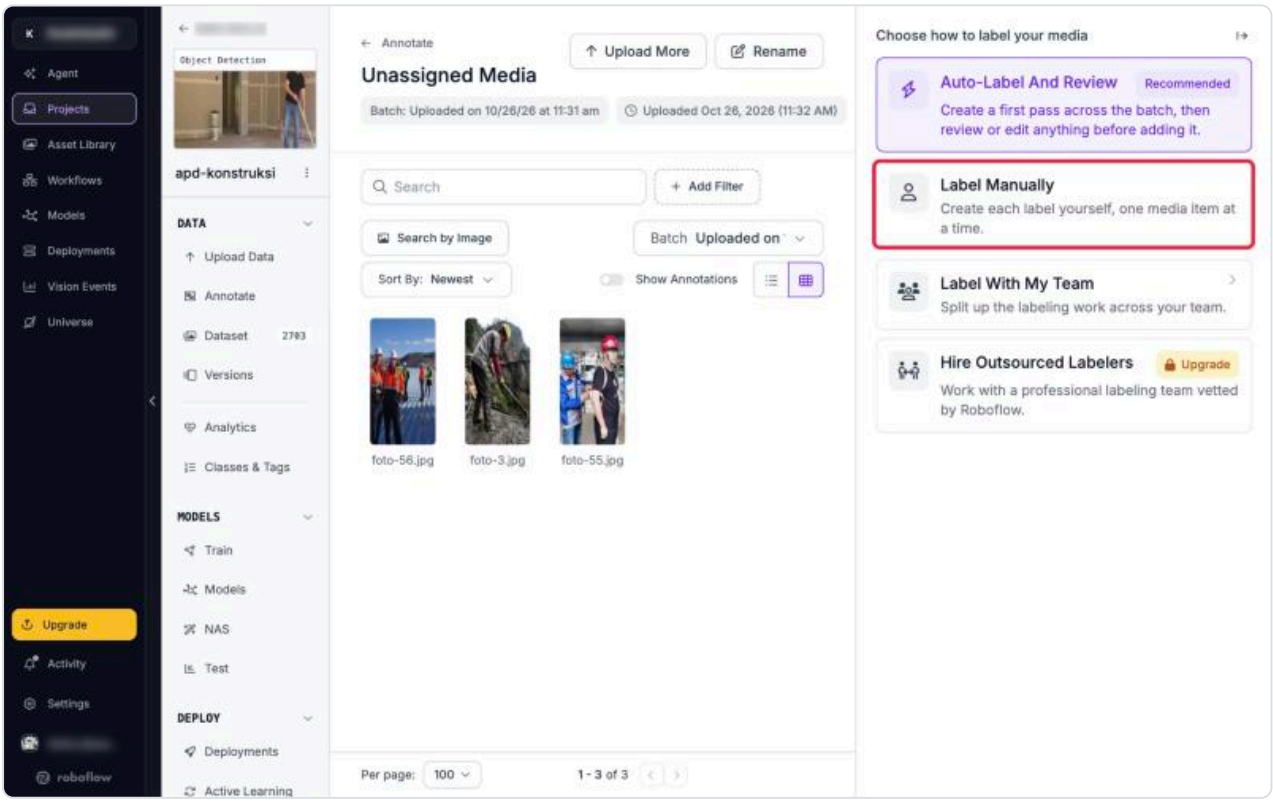
1. Buka project Anda di Roboflow, lalu menu **Upload Data**. Cara membuat project dijelaskan di Bab 5 Langkah 1.
2. Seret foto ke kotak upload. Tab **Not Annotated** menunjukkan jumlah foto yang belum punya label.
3. Klik **Save and Continue**.



Gambar 4.1 Halaman Upload Data berisi 3 foto tanpa label, tab Not Annotated dan tombol Save and Continue ditandai

Praktik langkah 2: Pilih cara melabeli

Setelah upload selesai, Roboflow menawarkan empat cara. Untuk latihan, pilih **Label Manually**.

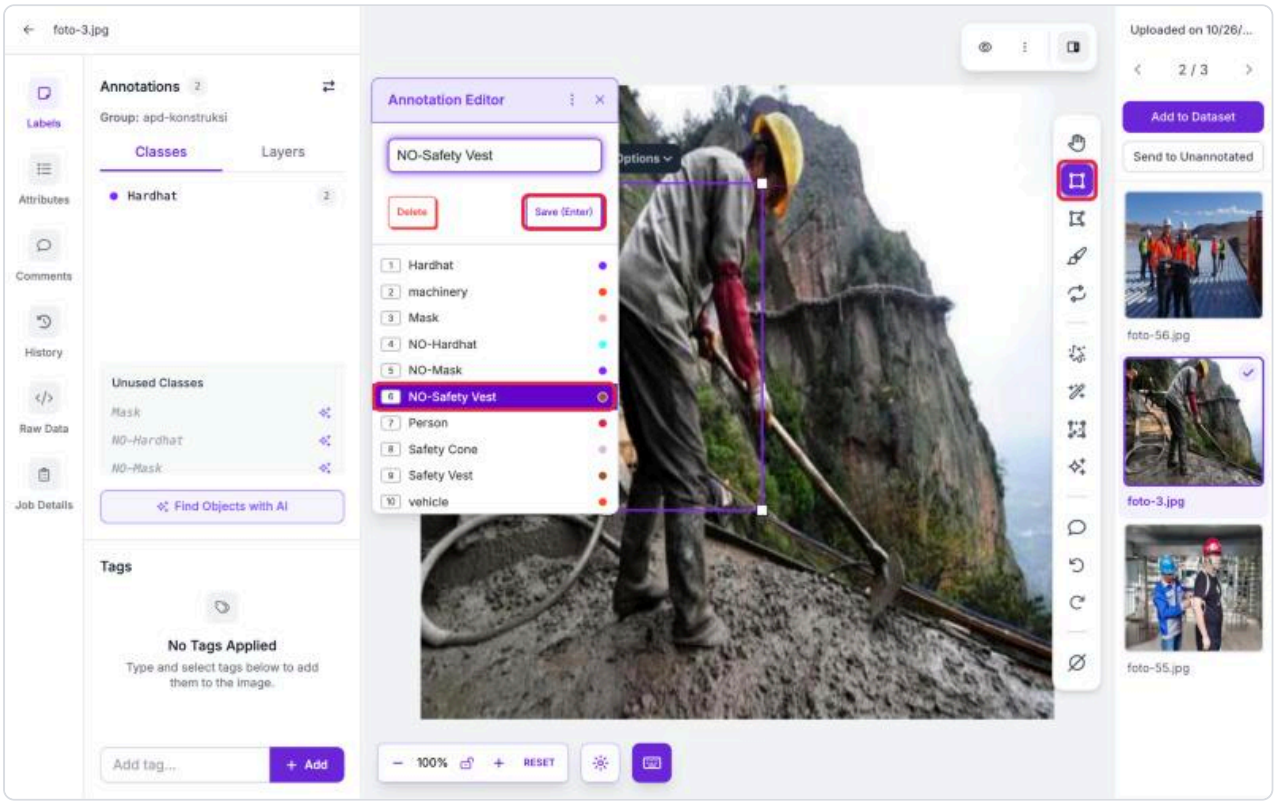


Gambar 4.2 Pilihan cara melabeli, Label Manually ditandai

Pilihan	Artinya
Auto-Label And Review	AI membuat label awal, lalu Anda memeriksanya. Memakai kredit akun
Label Manually	Anda menggambar sendiri setiap kotak
Label With My Team	Membagi foto ke beberapa anggota tim
Hire Outsourced Labelers	Membayar jasa labeling dari Roboflow

Praktik langkah 3: Gambar kotak dan pilih kelas

1. Klik ikon **Bounding Box** (gambar persegi) di toolbar kanan.
2. Tarik mouse dari pojok kiri atas objek ke pojok kanan bawahnya.
3. Jendela **Annotation Editor** muncul. Klik nama kelas yang benar di daftar, lalu klik **Save**.



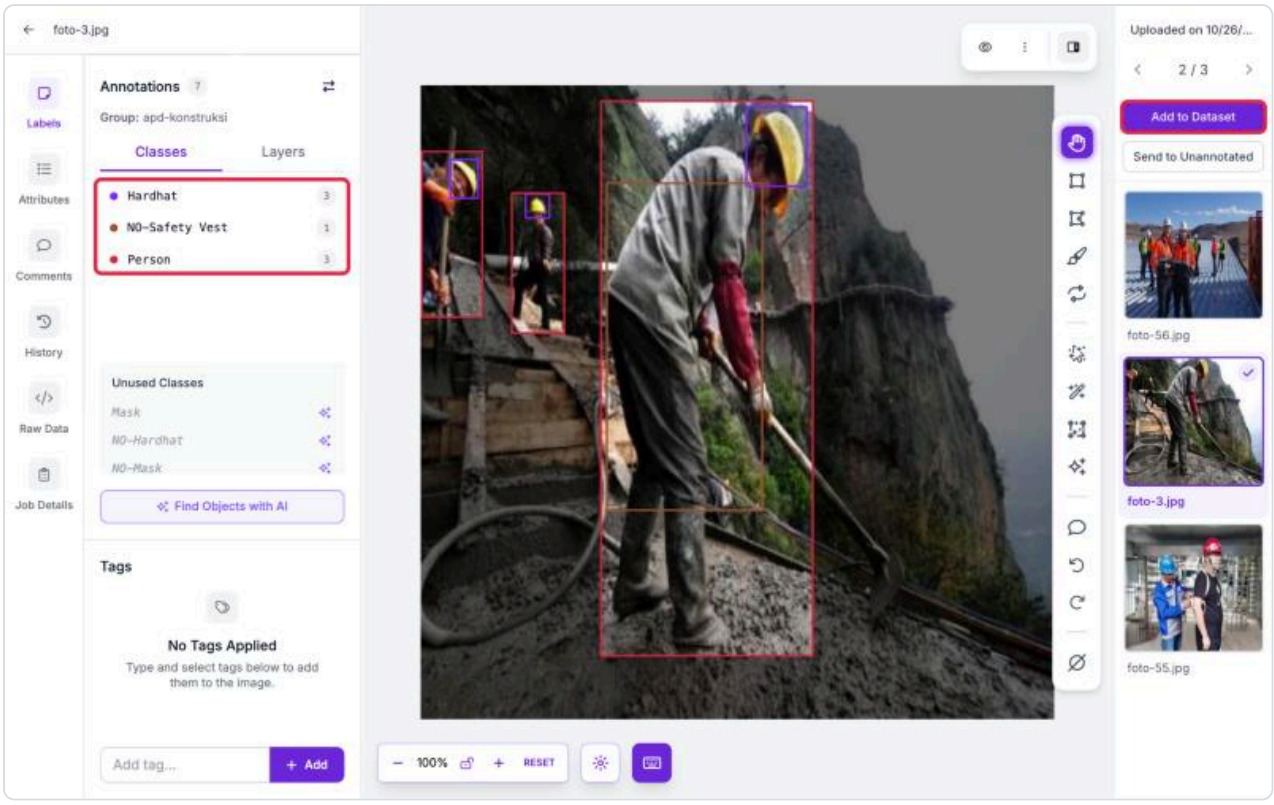
Gambar 4.3 Annotation Editor setelah kotak badan pekerja digambar, kelas NO-Safety Vest dipilih, tombol Save dan alat Bounding Box ditandai

Perhatikan dua hal di jendela ini:

- Roboflow otomatis memilih kelas yang terakhir Anda pakai. Selalu cek kelas yang disorot sebelum menyimpan.
- Klik nama kelas langsung di daftar. Mengetik di kolom teks saja tidak mengubah pilihan.

4. Ulangi untuk semua objek di foto. Panel **Annotations** di kiri menunjukkan jumlah label per kelas.

Foto contoh di bawah berisi 7 label: 3 Hardhat (termasuk dua pekerja kecil di belakang), 3 Person, dan 1 NO-Safety Vest untuk pekerja di depan yang tidak memakai rompi.

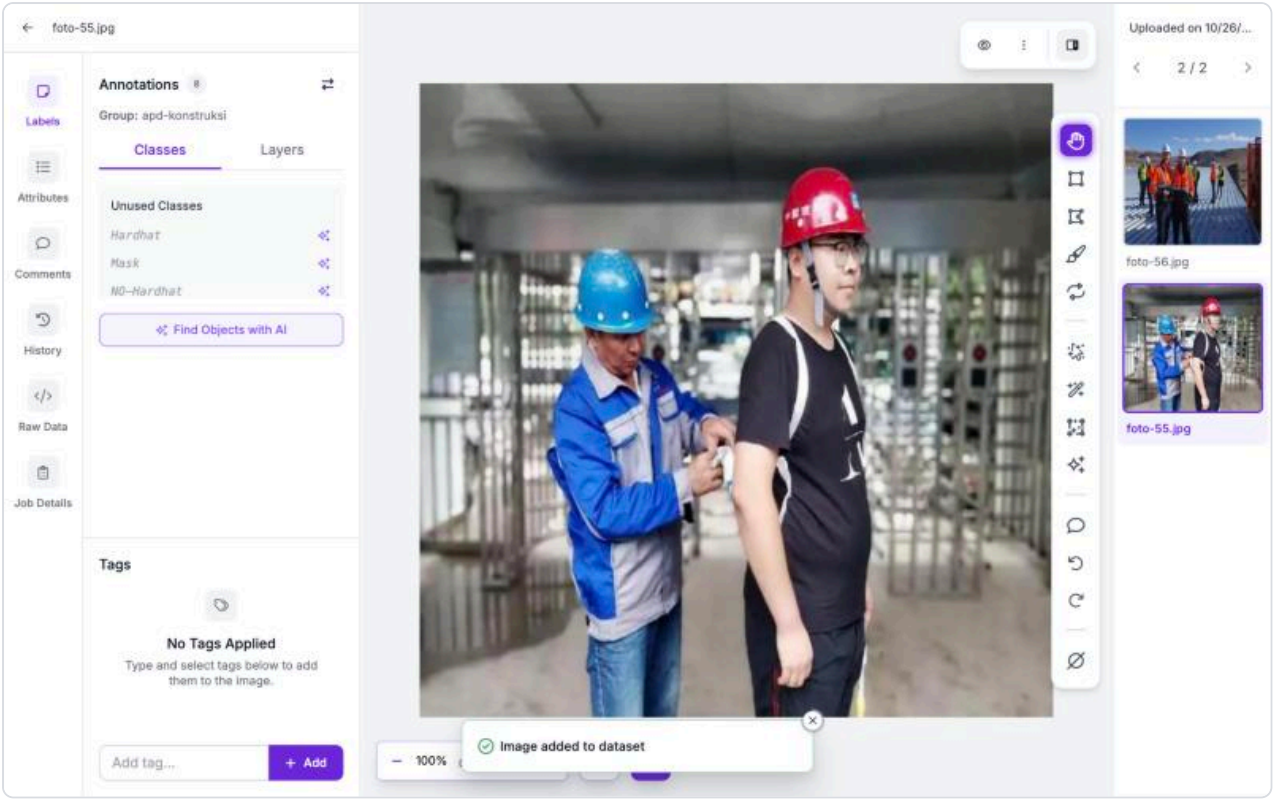


Gambar 4.4 Foto yang sudah dilabeli dengan 7 kotak, panel Annotations dan tombol Add to Dataset ditandai

Kalau ada kotak yang salah, pilih alat tangan (ikon paling atas di toolbar), klik kotaknya, lalu pilih kelas yang benar atau geser titik sudutnya.

Praktik langkah 4: Masukkan ke dataset

Klik **Add to Dataset** di kanan atas. Muncul pesan *Image added to dataset*, dan editor langsung membuka foto berikutnya.



Gambar 4.5 Pesan Image added to dataset dan editor berpindah ke foto berikutnya

Setelah semua foto selesai dilabeli, lanjutkan ke **Versions** dan **Train** seperti di Bab 5 Langkah 3 dan 4.

Cara mempercepat labeling

Labeling manual memakan waktu, rata-rata beberapa puluh detik per foto. Ada dua fitur yang membantu:

- **Auto Label dan Find Objects with AI.** AI mencari objek berdasarkan nama kelas dan menggambar kotak otomatis. Hasilnya tetap harus diperiksa, dan fitur ini memakai kredit akun. AI umum biasanya kesulitan dengan kelas "tidak memakai" seperti NO-Hardhat.
- **Label Assist.** Setelah project punya model terlatih, Roboflow bisa memakai model itu untuk menyarankan kotak di foto baru.

Strategi yang sering dipakai: labeli 100 sampai 200 foto secara manual, latih model pertama, pakai model itu untuk membantu melabeli foto berikutnya, periksa hasilnya, lalu latih lagi. Semakin sering diulang, semakin sedikit kotak yang perlu Anda perbaiki.

Mengekspor dataset untuk dilatih di Colab

Kalau ingin melatih dataset hasil labeling sendiri dengan notebook Colab (Bab 6):

1. Buka **Versions** dan buat versi dataset. Langkah ini memakai kredit akun.
2. Di halaman versi, klik **Download Dataset** dan pilih format **YOLOv11**.

3. Unduh sebagai file ZIP. Isinya folder `train`, `valid`, `test`, dan file `data.yaml`.
4. Di Colab, upload file ZIP lewat panel **Files** di sisi kiri, lalu ganti sel 2 dan 3 notebook dengan:

```
!unzip -q dataset.zip -d dataset
```

5. Di sel 5, ubah `data="data.yaml"` menjadi `data="dataset/data.yaml"`.

Nama tombol bisa sedikit berbeda karena Roboflow sering memperbarui tampilannya.

Aplikasi labeling gratis lainnya

Roboflow bukan satu-satunya pilihan. Dua aplikasi berikut gratis dan bisa mengekspor label ke format YOLO:

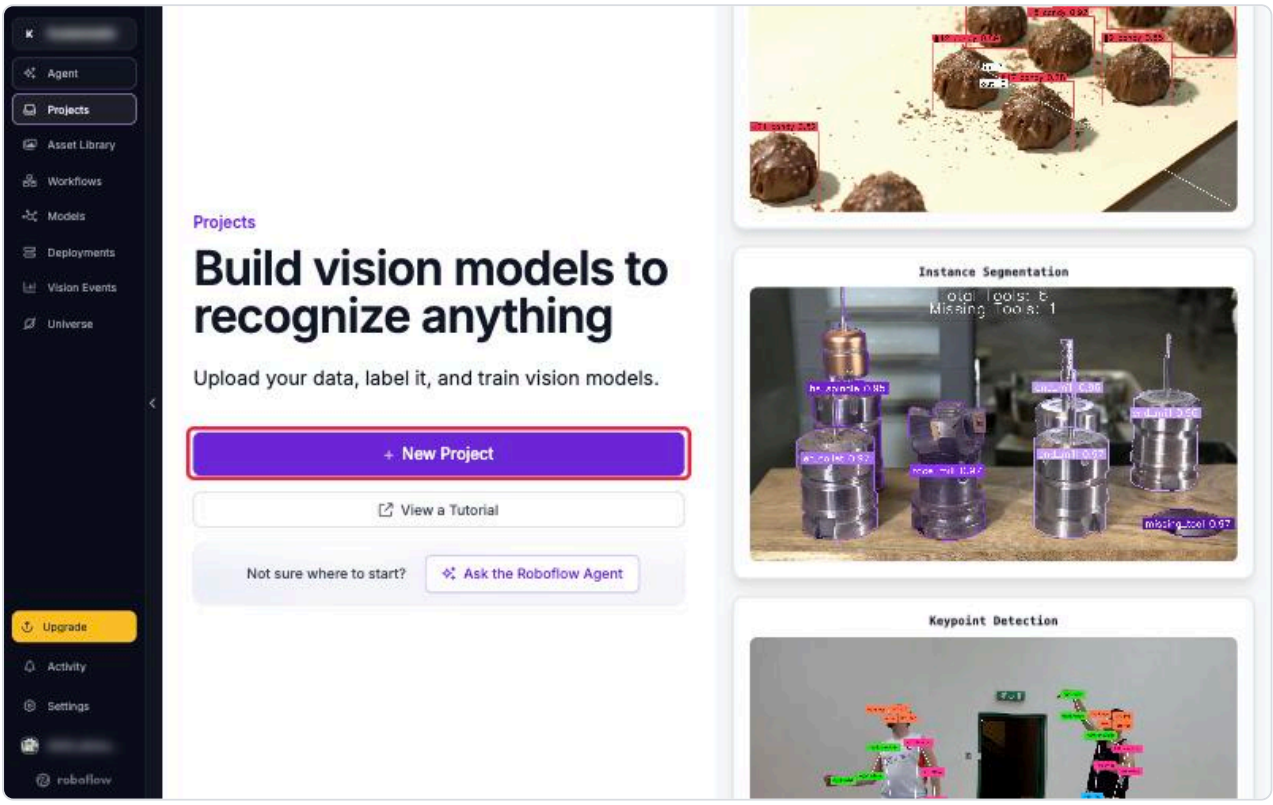
Aplikasi	Keterangan
CVAT	Bisa dipakai online atau di-install di server sendiri. Cocok untuk tim
Label Studio	Open source, dijalankan di laptop. Data tidak keluar dari komputer Anda

5. Jalur A: Training Cepat dengan Roboflow

Di jalur ini Anda melatih model lewat website Roboflow tanpa menulis kode. Hasil akhirnya dua hal: API Key dan Model ID, yang nanti Anda masukkan ke web di Bab 7. Roboflow cukup sering mengganti tampilan, jadi nama tombol bisa sedikit berbeda dari gambar.

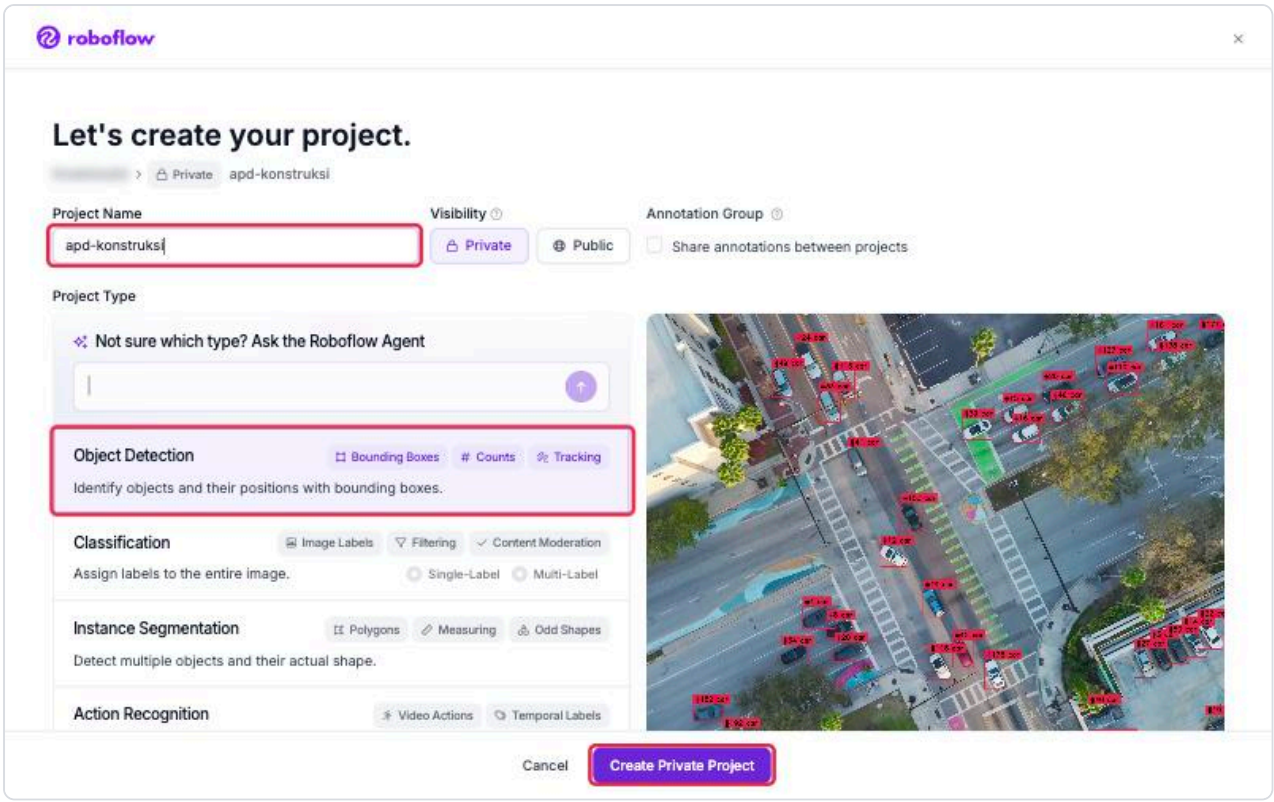
Langkah 1: Buat project

1. Login ke app.roboflow.com. Buat workspace kalau diminta.
2. Klik menu **Projects** di sisi kiri, lalu **New Project**.



Gambar 5.1 Halaman Projects di Roboflow, tombol New Project ditandai

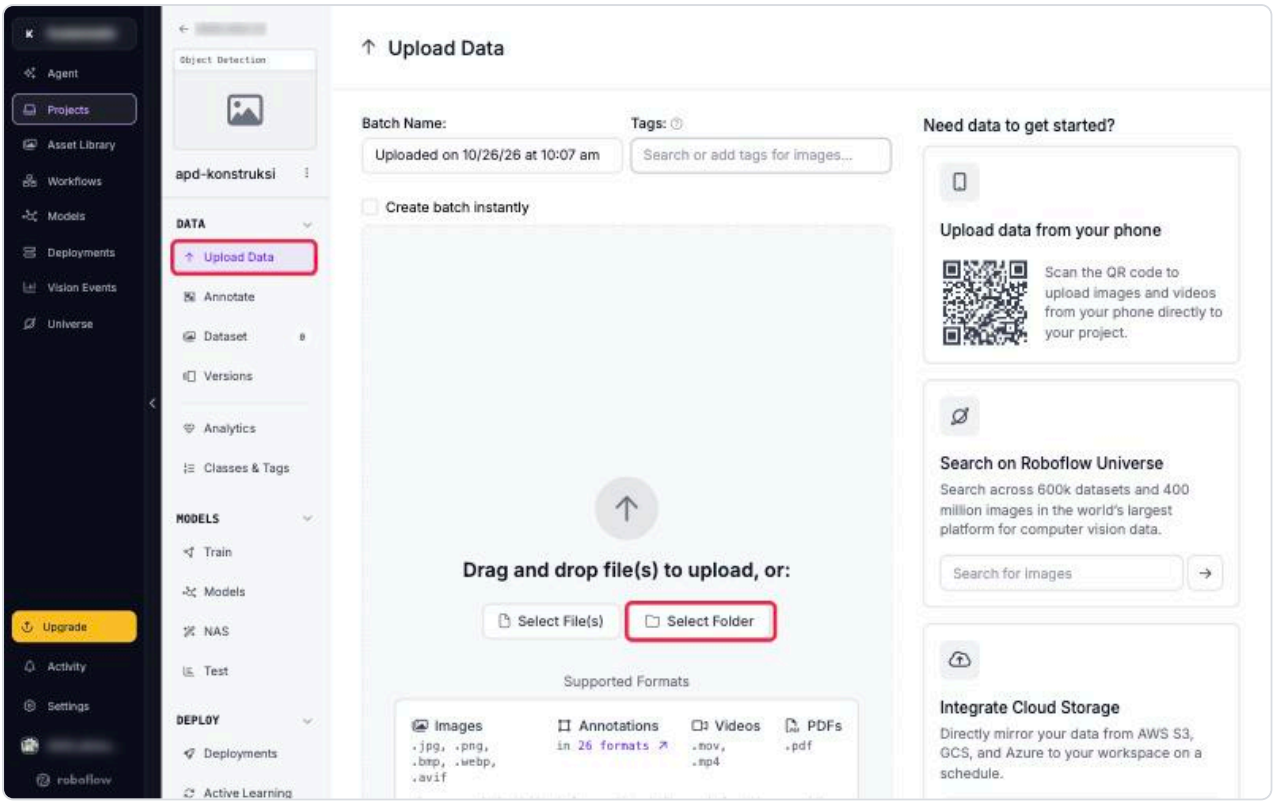
3. Isi **Project Name**, misalnya `apd-konstruksi`.
4. Pilih **Object Detection**, lalu klik **Create Private Project**.



Gambar 5.2 Form pembuatan project, kolom nama, pilihan Object Detection, dan tombol Create Private Project ditandai

Langkah 2: Upload dataset

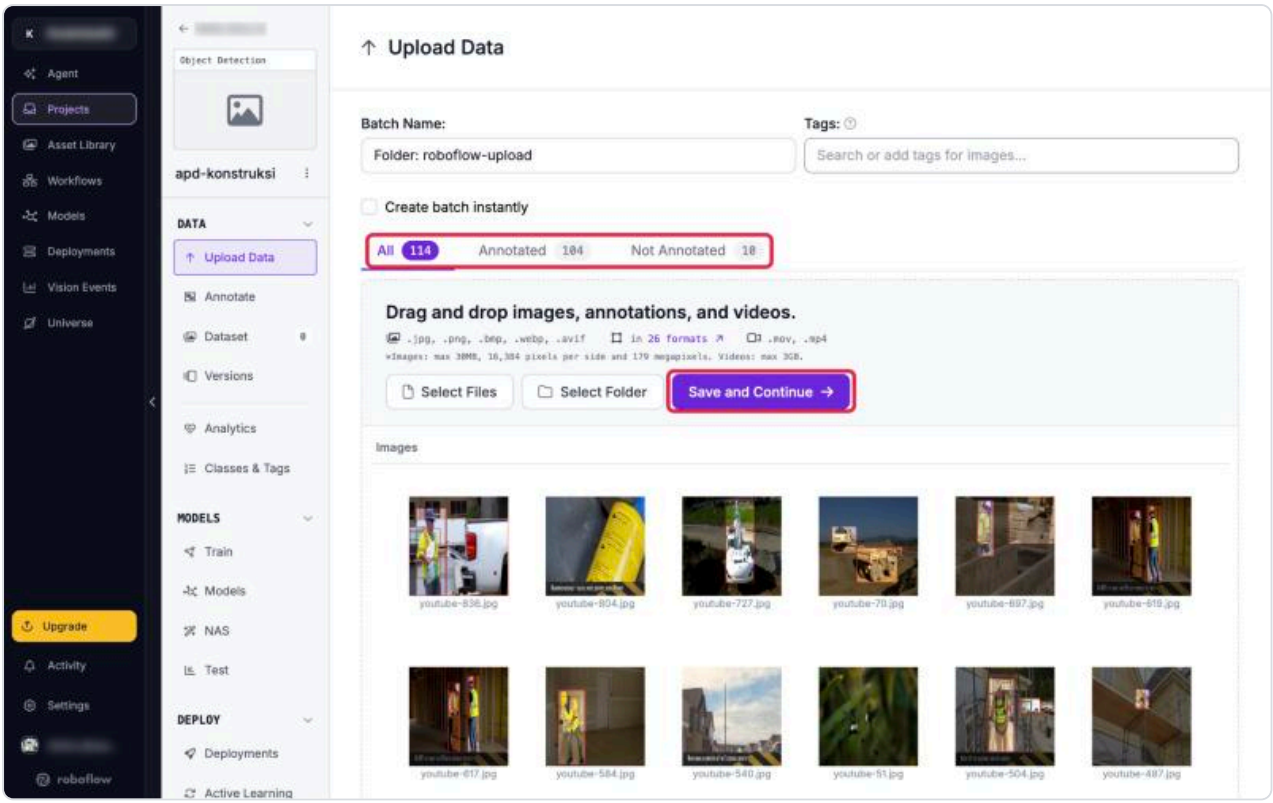
1. Setelah project dibuat, Roboflow langsung membuka halaman **Upload Data**.
2. Unduh dataset dari halaman Kaggle lewat tombol **Download** (lihat Bab 3), lalu ekstrak file ZIP-nya.
3. Klik **Select Folder** dan pilih folder `css-data`. Anda juga bisa menyeret folder itu ke kotak upload.



Gambar 5.3 Halaman Upload Data, menu Upload Data dan tombol Select Folder ditandai

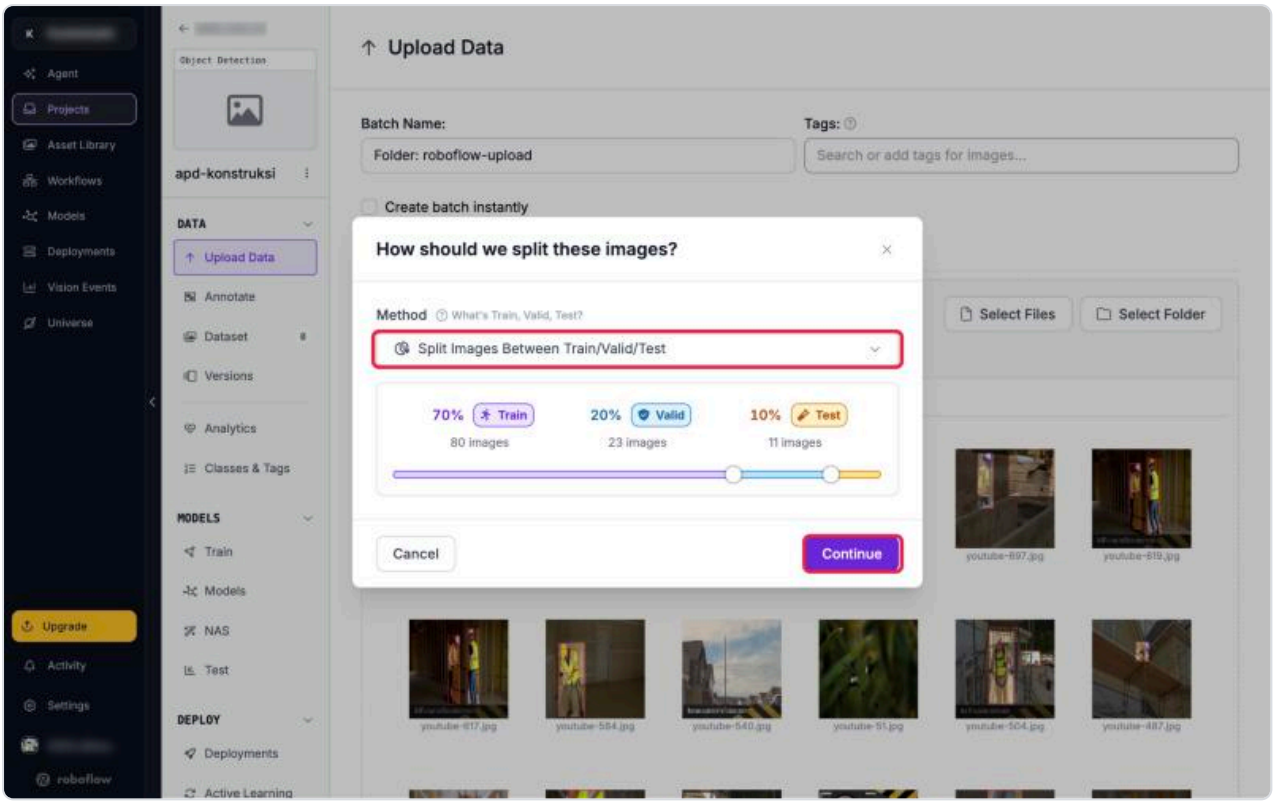
Pastikan file `data.yaml` ikut ter-upload. File label YOLO hanya berisi nomor kelas (0 sampai 9), sedangkan nama kelasnya ada di `data.yaml`. Kalau file itu tertinggal, Roboflow memakai angka sebagai nama kelas, sehingga muncul kelas bernama "0", "1", dan seterusnya. Cara memperbaikinya ada di Bab 11.

4. Tunggu sampai semua gambar terbaca. Di atas daftar gambar, Roboflow menampilkan jumlah gambar yang sudah punya label (Annotated). Klik **Save and Continue**.



Gambar 5.4 Daftar gambar yang siap di-upload, jumlah Annotated dan tombol Save and Continue ditandai

5. Roboflow lalu menanyakan cara membagi gambar. Kalau Anda meng-upload folder `css-data` lengkap, pilih **Use Existing Values** supaya pembagian train, valid, dan test bawaan dataset tetap dipakai. Kalau hanya meng-upload sebagian, pilih **Split Images Between Train/Valid/Test**. Klik **Continue**.

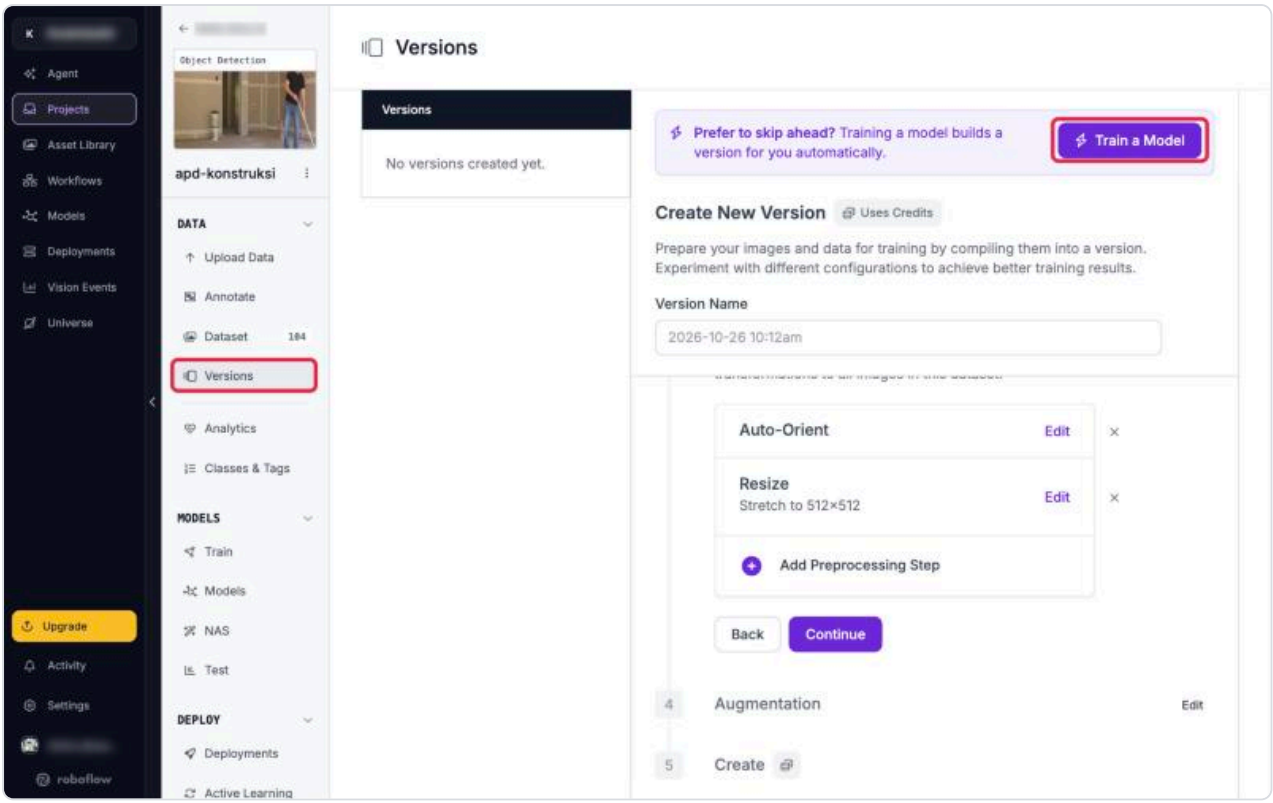


Gambar 5.5 Dialog pembagian gambar 70% train, 20% valid, 10% test, tombol Continue ditandai

Agar cepat saat workshop, cukup upload folder `valid` dan `test` ditambah sekitar 500 gambar dari `train`. Upload 2.801 gambar sekaligus bisa memakan waktu lama di jaringan kelas.

Langkah 3: Buka halaman training

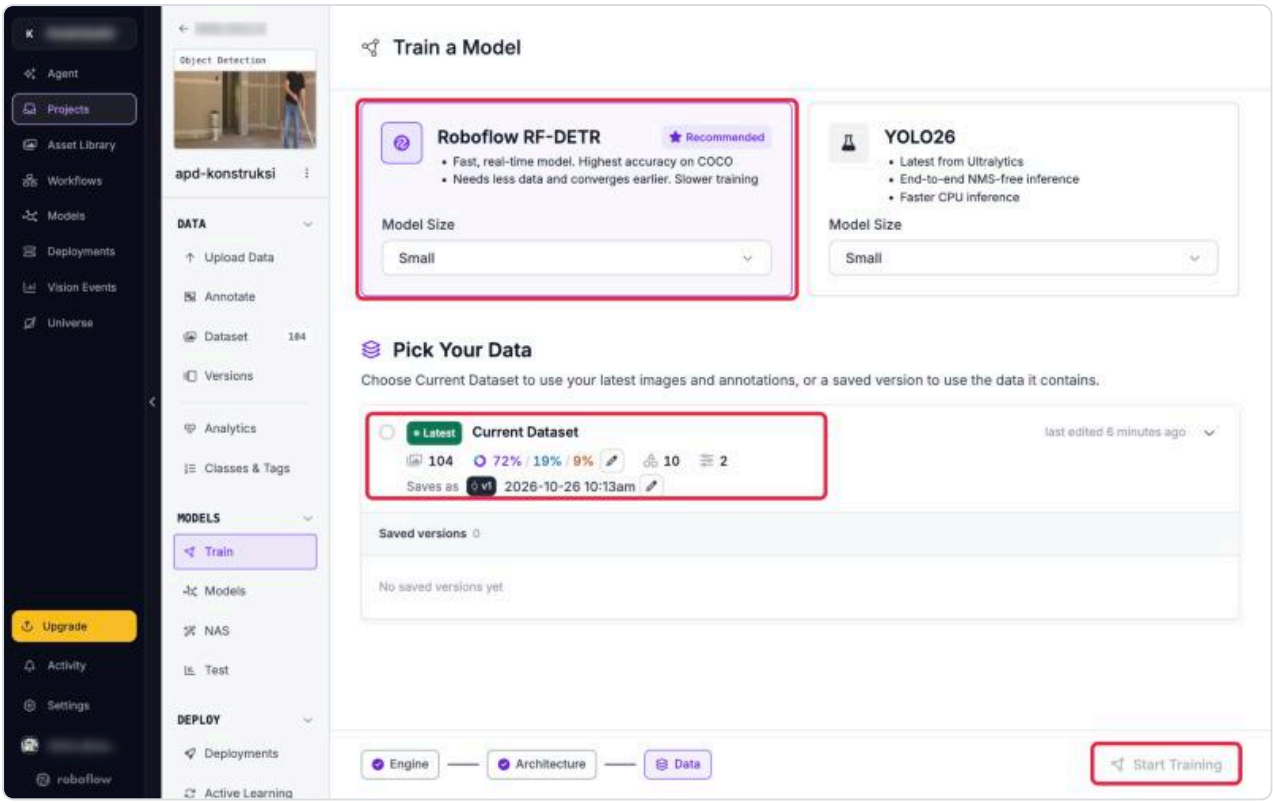
1. Klik menu **Versions** di sisi kiri.
2. Di halaman ini Anda bisa menyusun versi dataset sendiri. Cara yang lebih cepat: klik **Train a Model**, dan Roboflow membuat versi dataset secara otomatis.



Gambar 5.6 Halaman Versions, menu Versions dan tombol Train a Model ditandai

Langkah 4: Mulai training

1. Pilih **Custom Training**.
2. Pilih arsitektur model. Roboflow menyarankan **Roboflow RF-DETR** dengan ukuran Small.
3. Di bagian **Pick Your Data**, pilih **Current Dataset**.
4. Klik **Start Training**. Roboflow mengirim email kalau training sudah selesai.



Gambar 5.7 Halaman Train a Model, pilihan RF-DETR, Current Dataset, dan tombol Start Training ditandai

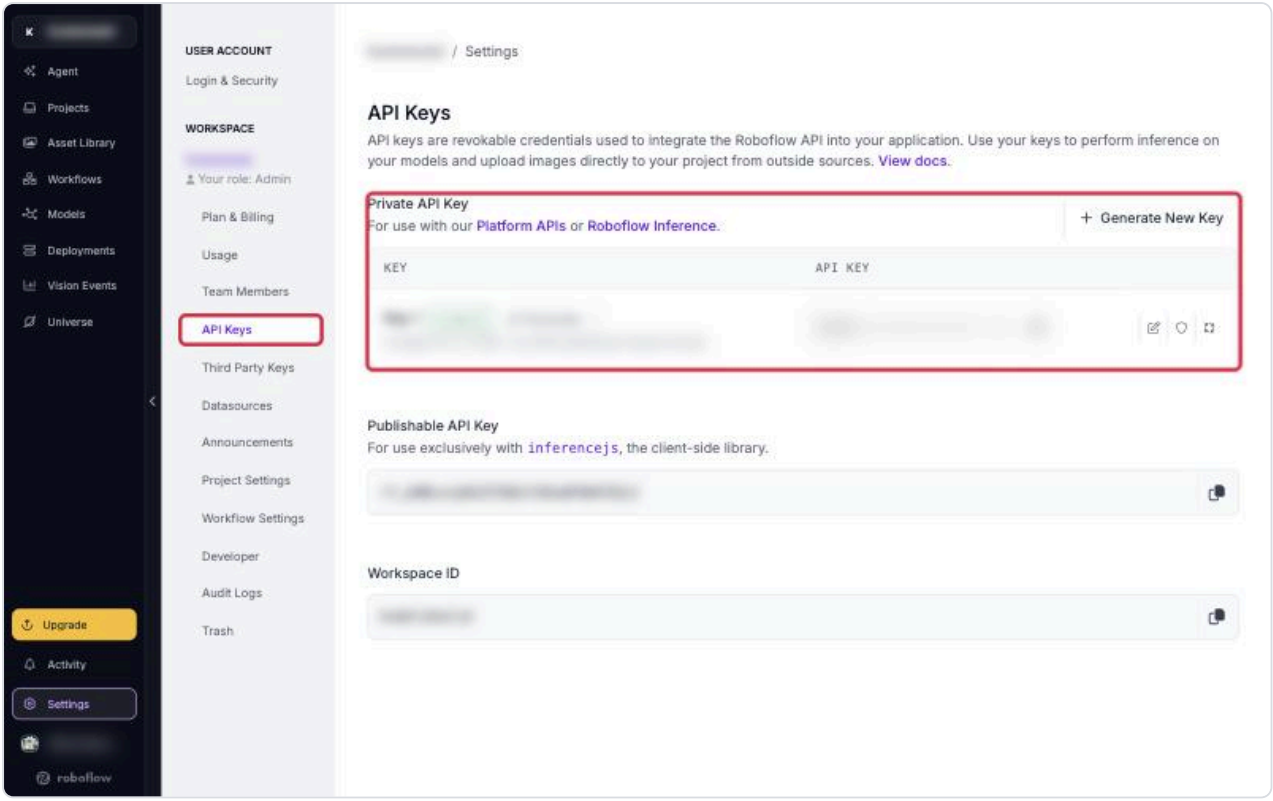
Akun gratis Roboflow punya kuota training terbatas. Cek sisa kuota di **Settings > Usage** sebelum hari workshop.

Langkah 5: Coba model di browser

Setelah training selesai, buka tab **Visualize** atau **Deployments**. Upload satu foto konstruksi dan lihat kotak deteksinya muncul langsung di halaman.

Langkah 6: Salin API Key dan Model ID

- **Model ID** formatnya `nama-project/nomor-versi`, contohnya `apd-konstruksi/1`. Anda bisa melihatnya di halaman model atau di contoh kode deploy.
- **API Key**: klik **Settings** di kiri bawah, lalu **API Keys**. Salin kunci di bagian **Private API Key** lewat ikon salin di sebelah kanannya.



Gambar 5.8 Halaman Settings, menu API Keys dan bagian Private API Key ditandai. Kunci sengaja diburamkan

Simpan API Key untuk diri sendiri. Jangan menempelkannya di kode yang akan di-upload ke GitHub.

Kalau training belum selesai

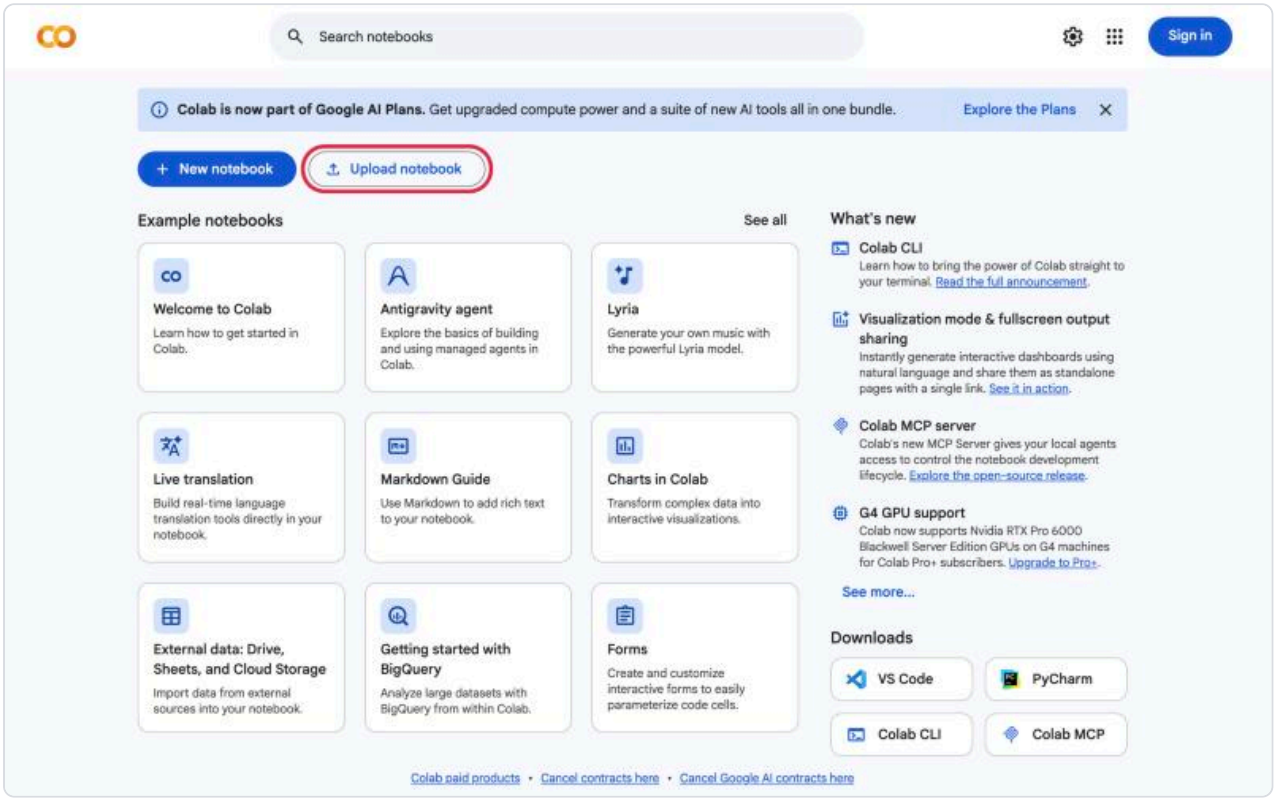
Cari "construction site safety" di Roboflow Universe. Banyak project publik di sana yang sudah punya model terlatih. Buka salah satunya dan salin Model ID dari bagian Model atau contoh kode API-nya.

6. Jalur B: Training Model Sendiri di Google Colab

Di jalur ini Anda melatih YOLO11 di Google Colab dan pulang membawa file `best.pt`. Colab meminjamkan GPU gratis, jadi laptop dengan spesifikasi biasa pun bisa ikut.

Langkah 1: Upload notebook ke Colab

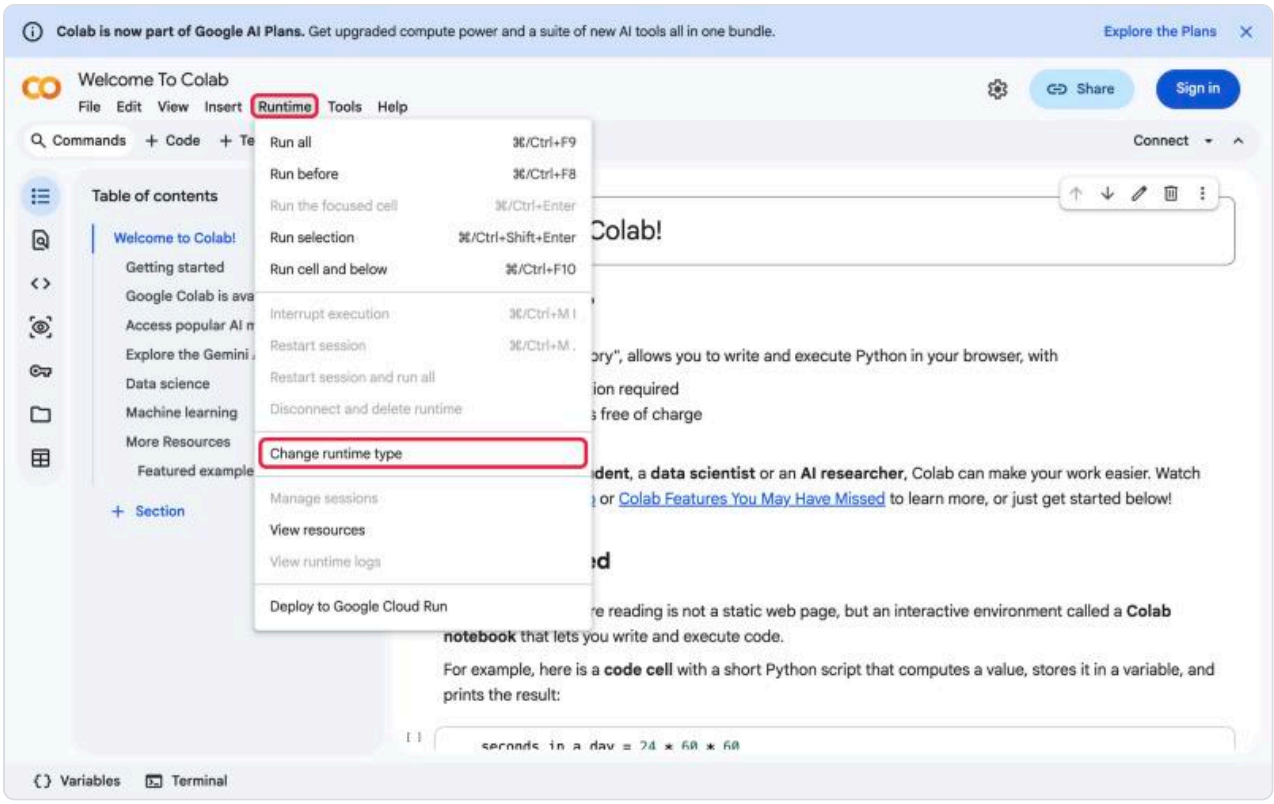
1. Buka colab.research.google.com dan login dengan akun Google.
2. Klik **Upload notebook**, lalu pilih file `notebooks/training_yolo_colab.ipynb` dari folder proyek.



Gambar 6.1 Halaman awal Google Colab, tombol Upload notebook ditandai kotak merah

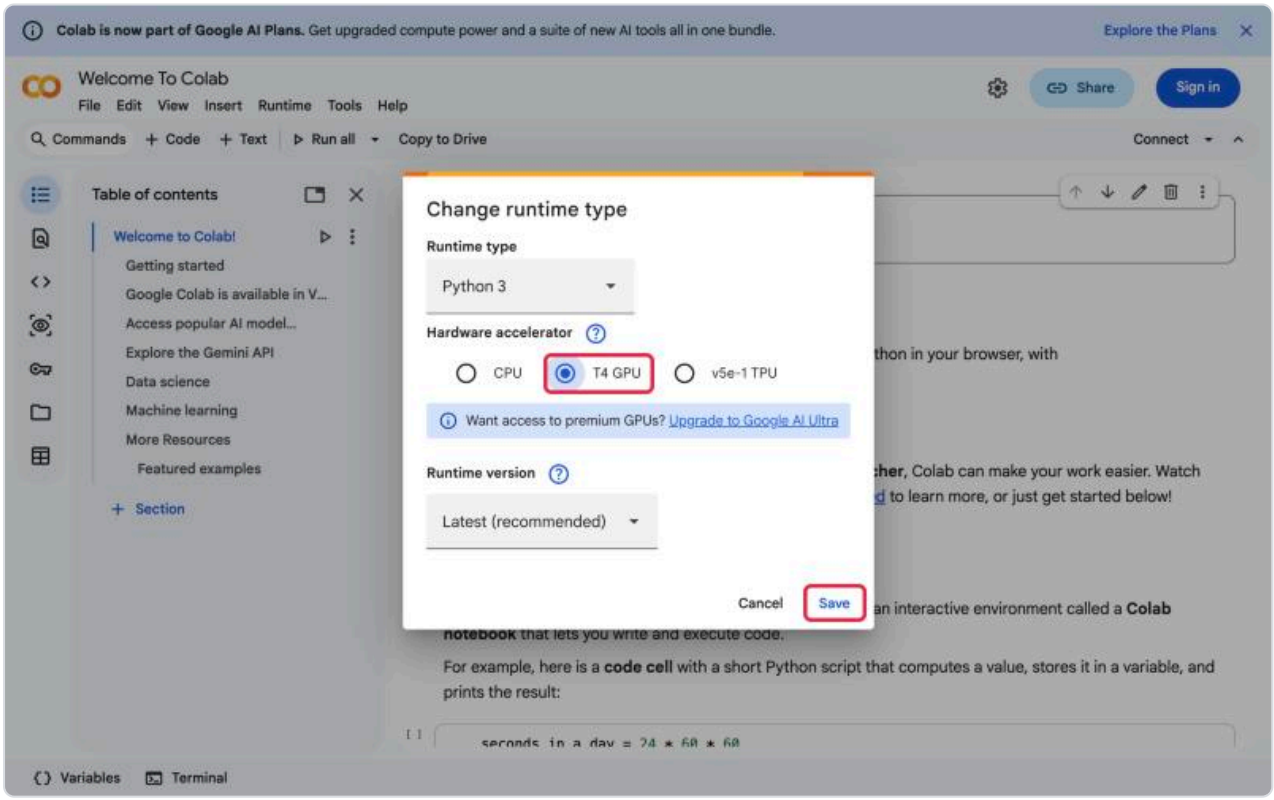
Langkah 2: Aktifkan GPU

1. Klik menu **Runtime**, lalu **Change runtime type**.



Gambar 6.2 Menu Runtime terbuka, pilihan Change runtime type ditandai

2. Pilih **T4 GPU**, lalu klik **Save**.



Gambar 6.3 Dialog Change runtime type dengan T4 GPU terpilih dan tombol Save ditandai

Tanpa GPU, training yang biasanya 40 menit bisa berjalan berjam-jam.

Langkah 3: Jalankan sel satu per satu

Klik tombol play di kiri setiap sel, atau tekan `Shift + Enter`. Tunggu sel selesai sebelum lanjut ke sel berikutnya.

Sel	Isi	Waktu
1	Install `ultralytics` dan `kagglehub`	1 menit
2	Unduh dataset dari Kaggle	1 menit
3	Buat file `data.yaml`	Instan
4	Tampilkan 3 contoh gambar	Instan
5	Training 30 epoch	30 sampai 40 menit
6	Tampilkan grafik hasil dan confusion matrix	Instan
7	Uji model pada gambar test	1 menit
8	Unduh `best.pt`	Instan

Isi kode setiap sel

Anda tidak perlu mengetik ulang kode ini karena sudah ada di notebook. Bagian ini menjelaskan apa yang dikerjakan setiap sel.

Sel 1: install library

```
!pip install -q ultralytics kagglehub
```

Tanda `!` di depan berarti perintah ini dijalankan seperti di terminal, bukan sebagai kode Python. `pip install` memasang library, dan `-q` (quiet) membuat tulisan prosesnya lebih sedikit. `ultralytics` berisi YOLO, sedangkan `kagglehub` dipakai untuk mengunduh dataset.

Sel 2: unduh dataset

```
import kagglehub

folder = kagglehub.dataset_download("snehilsanyal/construction-site-safety-image-dataset-roboflow")
dataset = folder + "/css-data"
print(dataset)
```

Teks di dalam tanda kutip adalah alamat dataset di Kaggle (nama pembuat/nama dataset). Fungsi `dataset_download` mengunduh dan mengekstrak dataset, lalu memberi tahu letak foldernya. Baris berikutnya menambahkan `/css-data` karena gambar dan label ada di dalam subfolder itu.

Sel 3: file konfigurasi

```
kelas = ["Hardhat", "Mask", "NO-Hardhat", "NO-Mask", "NO-Safety Vest",
         "Person", "Safety Cone", "Safety Vest", "machinery", "vehicle"]
```

Urutan nama di daftar ini harus sama persis dengan nomor kelas di file label: nama pertama untuk nomor 0, nama kedua untuk nomor 1, dan seterusnya. Kalau urutannya tertukar, model akan menyebut helm sebagai masker. Sisa kode di sel ini menulis lokasi folder dan daftar kelas ke file `data.yaml`.

Sel 7: uji model

```
model = YOLO(f"{hasil}/weights/best.pt")
metrics = model.val(data="data.yaml", split="test")
hasil_prediksi = model(gambar)
```

Baris pertama memuat model terbaik hasil training. `model.val(..., split="test")` menguji model dengan gambar test dan menghitung mAP. Baris ketiga mencoba model pada satu gambar, lalu hasilnya disimpan dan ditampilkan.

Sel 8: unduh model

```
from google.colab import files
files.download(f"{hasil}/weights/best.pt")
```

Perintah ini hanya ada di Google Colab. Browser akan mengunduh `best.pt` ke folder Downloads di laptop Anda.

Kode training

Seluruh proses training sebenarnya hanya tiga baris:

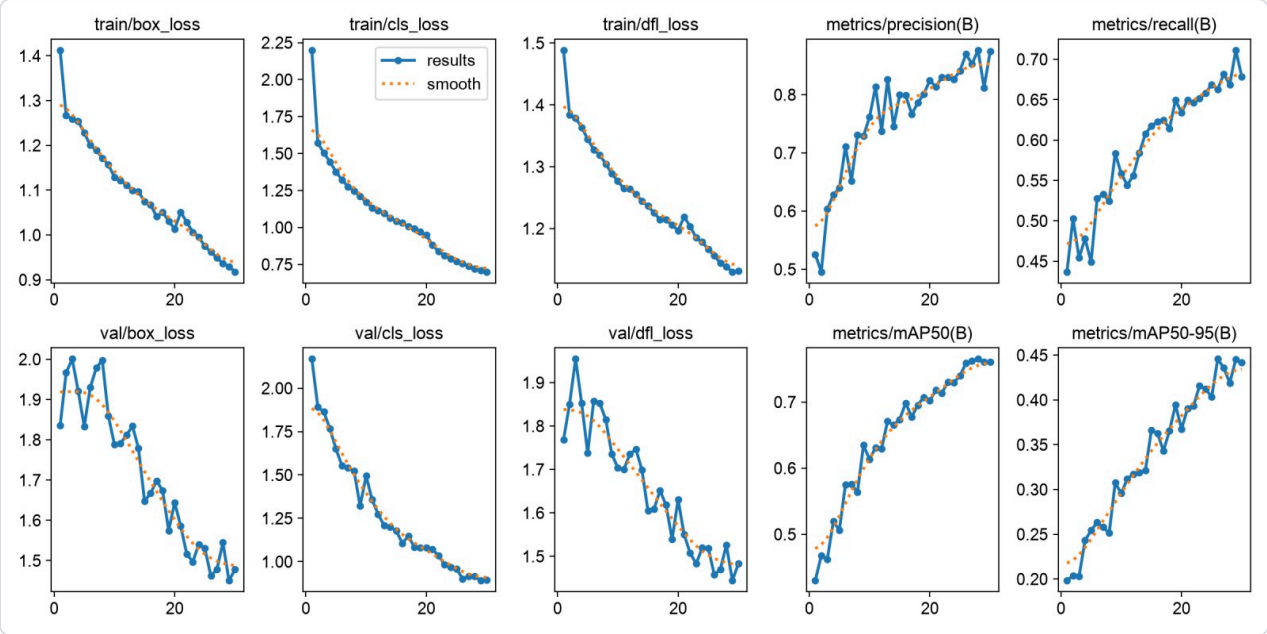
```
from ultralytics import YOLO

model = YOLO("yolo11n.pt")
model.train(data="data.yaml", epochs=30, imgsz=640)
```

`yolo11n.pt` adalah model dasar yang sudah mengenal benda sehari-hari seperti orang dan mobil. Kita melatihnya ulang supaya mengenal APD. `epochs=30` berarti model membaca seluruh gambar training 30 kali, dan `imgsz=640` mengecilkan setiap gambar ke 640×640 piksel.

Langkah 4: Baca hasil training

Sel 6 menampilkan dua gambar. Yang pertama, `results.png`, berisi grafik loss dan akurasi untuk setiap epoch.

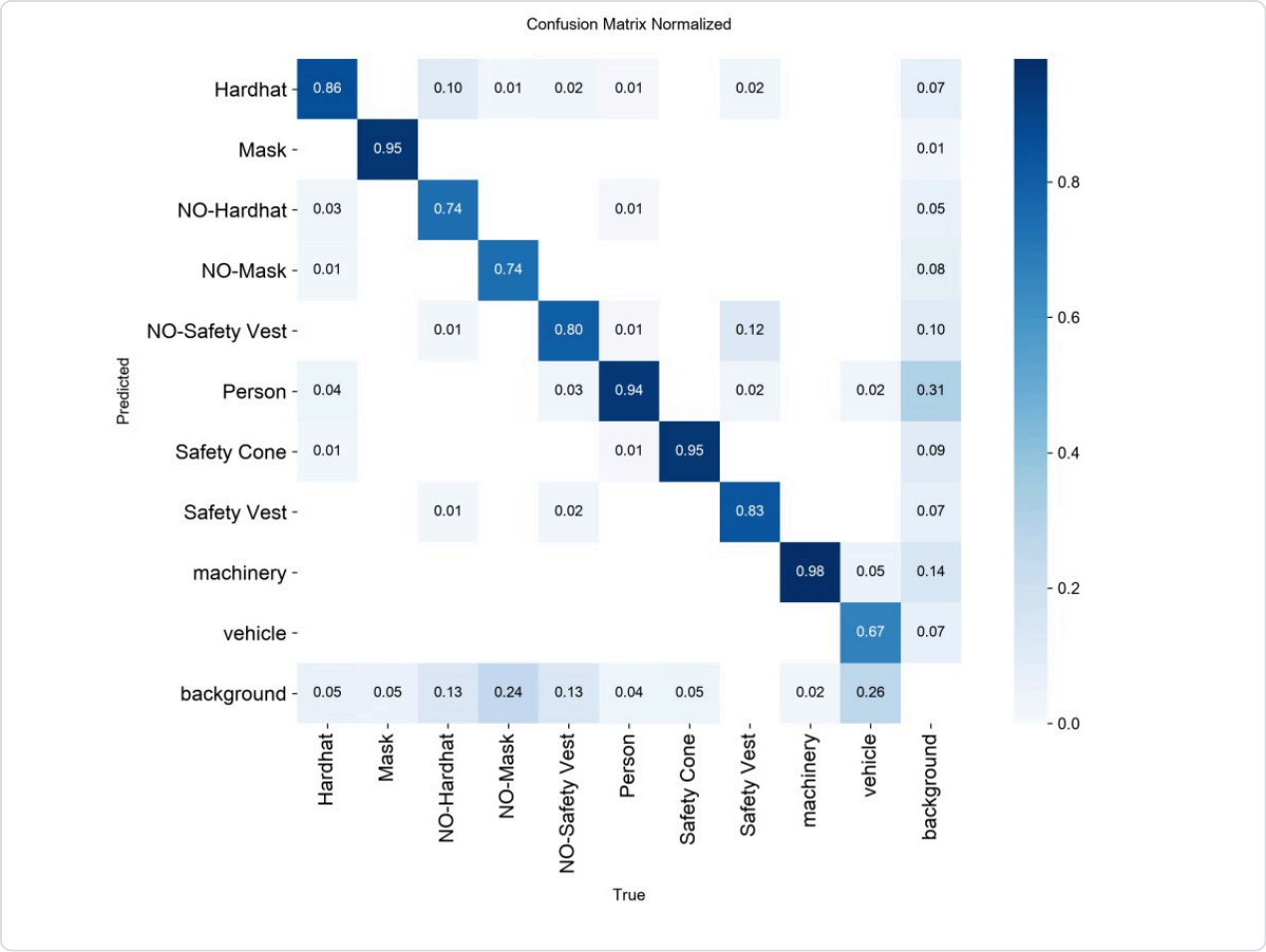


Gambar 6.4 Grafik results.png: loss train dan valid turun, precision, recall, dan mAP naik selama 30 epoch

- Baris atas dan bawah sebelah kiri adalah **loss**, ukuran kesalahan model. Grafik ini sebaiknya terus turun.
- Dua grafik paling kanan adalah **mAP50** dan **mAP50–95**, nilai akurasi dari 0 sampai 1. Grafik ini sebaiknya terus naik.

Grafik di atas berasal dari training untuk buku ini (30 epoch). Loss masih turun dan mAP50 masih naik di epoch terakhir, jadi model ini sebenarnya masih bisa lebih baik kalau epoch ditambah, misalnya menjadi 50.

Gambar kedua adalah confusion matrix. Kolom menunjukkan kelas yang sebenarnya, baris menunjukkan tebakan model. Angka di garis diagonal adalah persentase tebakan yang benar.



Gambar 6.5 Confusion matrix ternormalisasi untuk 10 kelas dan background

Dari confusion matrix ini terlihat dua kelemahan model. Sebanyak 24% orang tanpa masker (NO-Mask) tidak terdeteksi sama sekali, dan 10% orang tanpa helm (NO-Hardhat) terbaca sebagai memakai helm.

Sel 7 menguji model dengan 82 gambar test yang belum pernah dilihat saat training. Model kita mendapat **mAP50 0,70**. Kelas yang paling mudah dikenali adalah Hardhat (0,88), sedangkan yang paling sulit adalah Safety Cone (0,43) dan NO-Hardhat (0,51).

Langkah 5: Simpan model

Sel terakhir mengunduh `best.pt` ke laptop. Pindahkan file itu ke folder `models/` di proyek.

Pilihan lain: training di laptop

Cara ini butuh laptop yang cukup kuat. Tanpa GPU, prosesnya bisa beberapa jam.

```
python download_data.py
python train.py
```

Pemilik Mac M1 atau yang lebih baru bisa mempercepat training dengan menghapus tanda `#` di depan baris `device="mps"` dalam `train.py`. Model hasilnya ada di `runs/detect/apd/weights/best.pt`. Salin ke folder `models/`.

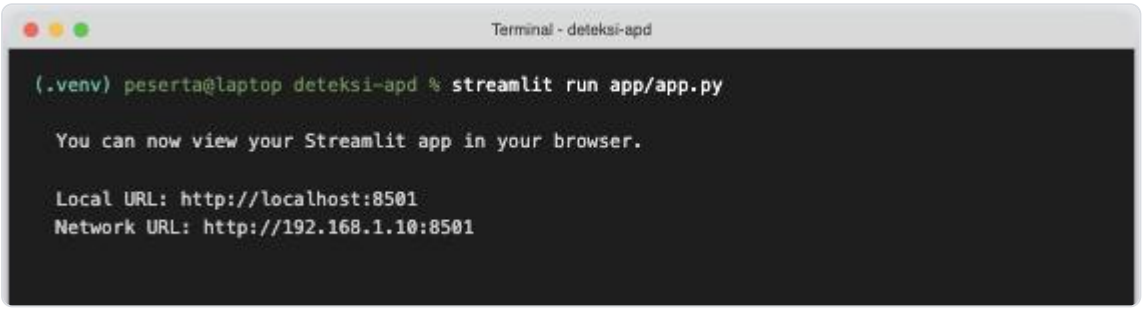
7. Menjalankan dan Menguji Web Deteksi

Web kita dibuat dengan Streamlit, library yang mengubah kode Python biasa menjadi halaman web. Semua kodenya ada di satu file, `app/app.py`.

Langkah 1: Jalankan web

Pastikan `best.pt` sudah ada di folder `models/` dan tulisan `(.venv)` muncul di terminal. Setelah itu ketik:

```
streamlit run app/app.py
```

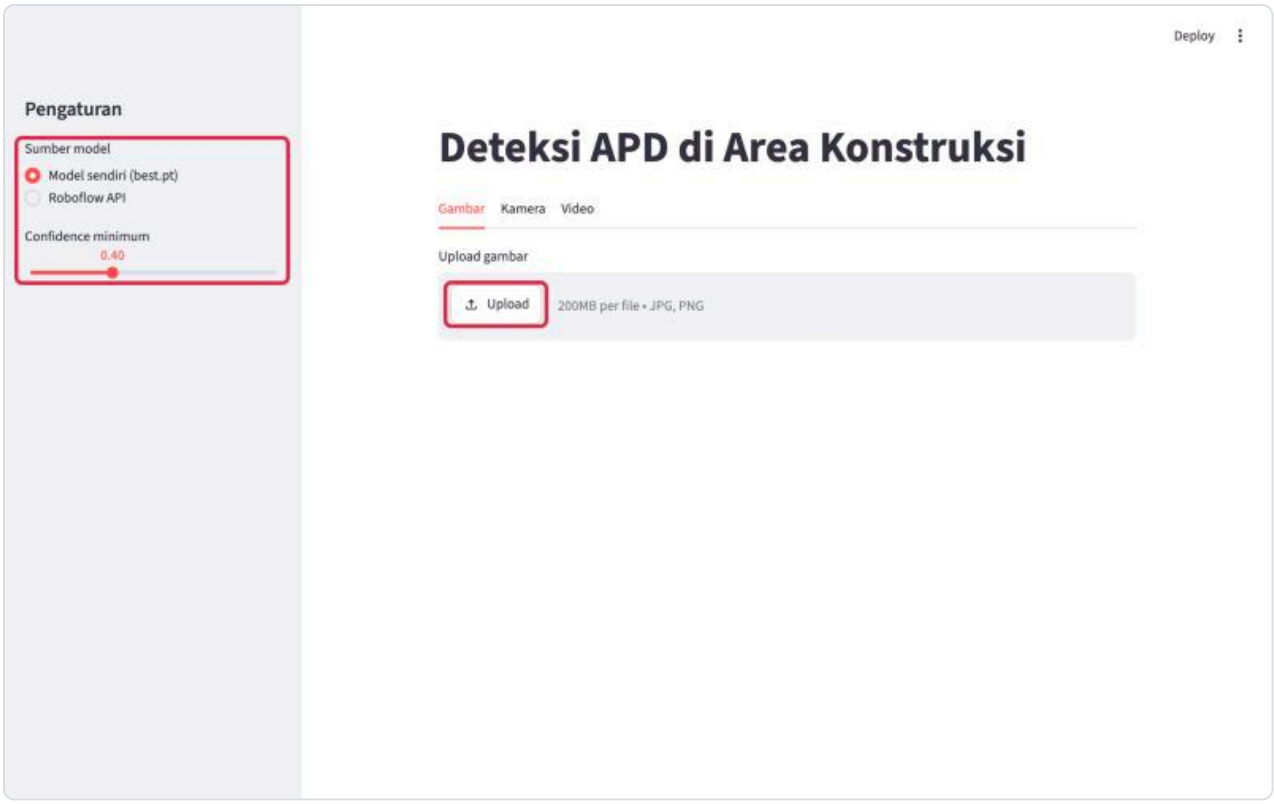


Gambar 7.1 Terminal menampilkan alamat Local URL http://localhost:8501 setelah streamlit run

Browser akan terbuka sendiri di `http://localhost:8501`. Kalau tidak, salin alamat Local URL dari terminal ke browser. Untuk mematikan web, kembali ke terminal dan tekan `Ctrl + C`.

Langkah 2: Kenali tampilannya

Di sebelah kiri ada pengaturan. Di sebelah kanan ada tiga tab: Gambar, Kamera, dan Video.

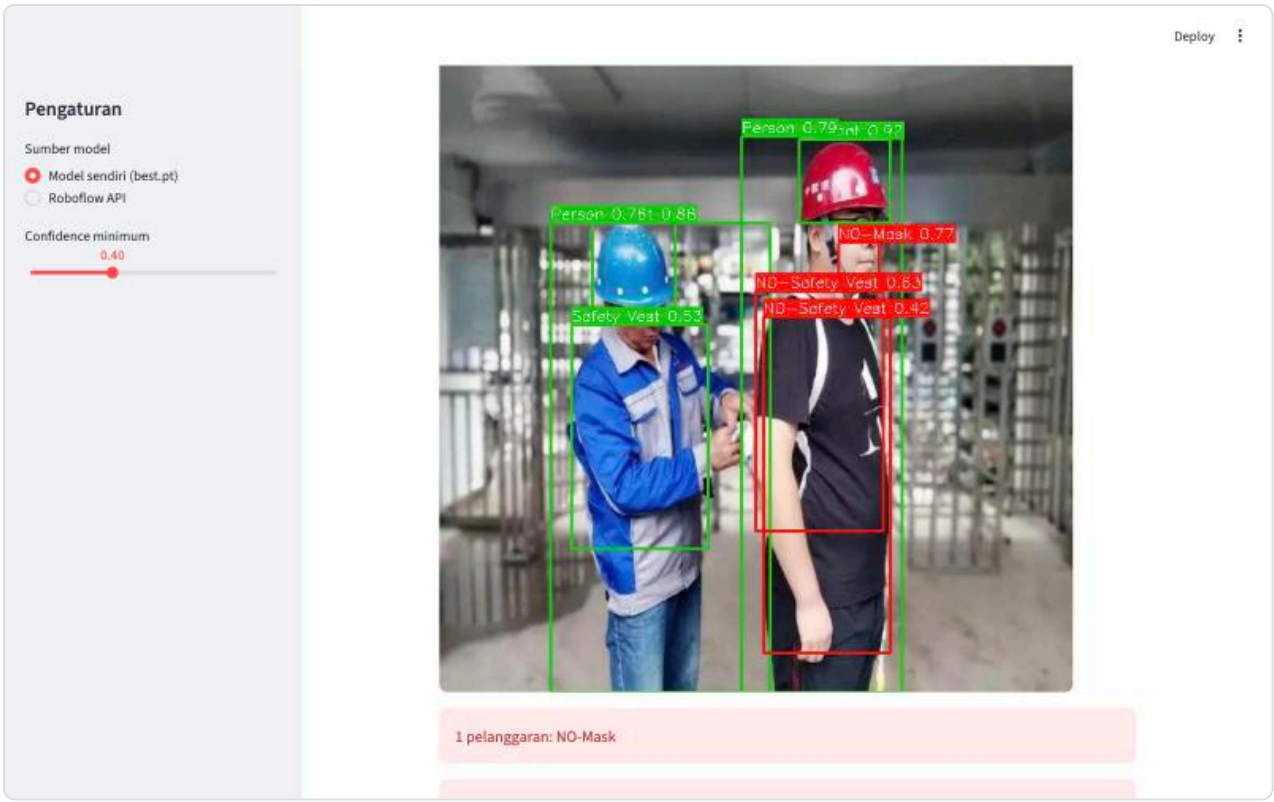


Gambar 7.2 Tampilan awal web, pengaturan di sidebar dan tombol Upload ditandai

- **Sumber model**: pilih Model sendiri (best.pt) untuk hasil Jalur B, atau Roboflow API untuk hasil Jalur A.
- **Confidence minimum**: model hanya menampilkan objek yang keyakinannya di atas angka ini. Nilai 0.40 berarti 40%.

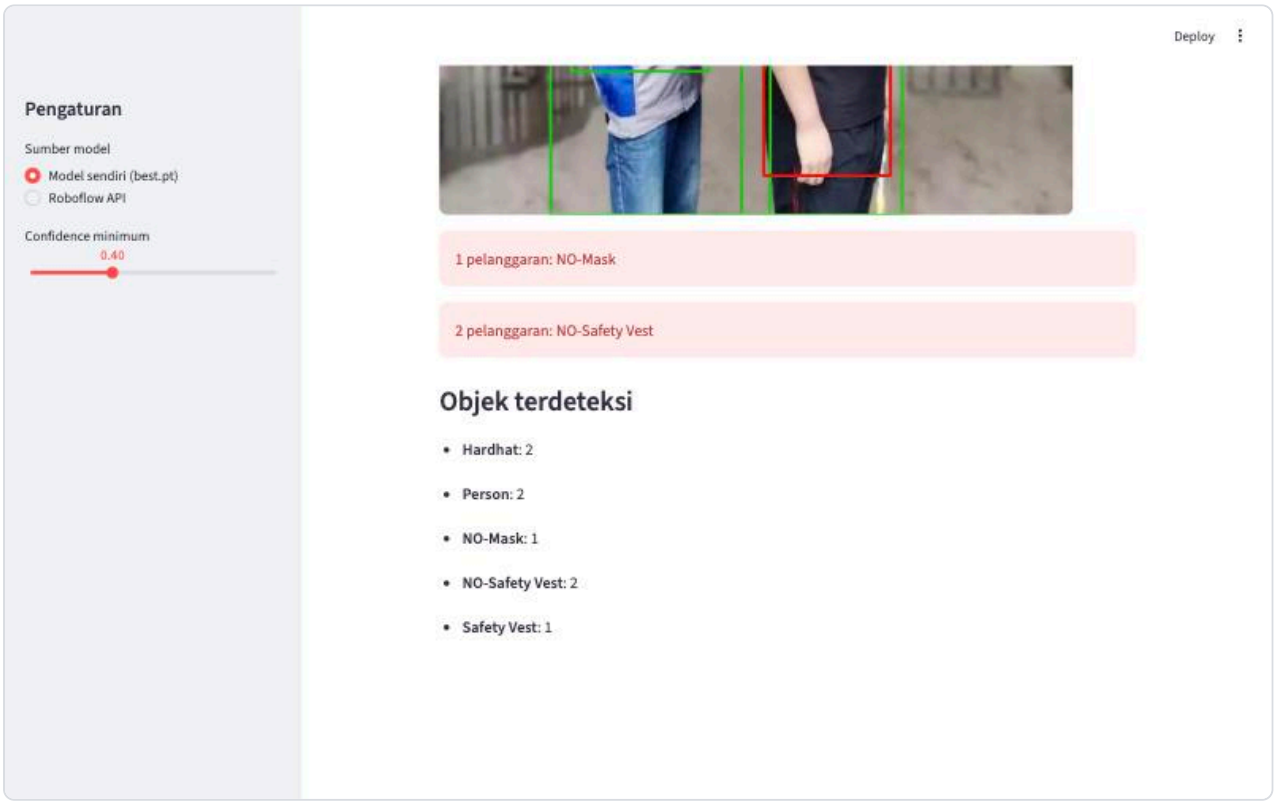
Langkah 3: Deteksi dari foto

1. Buka tab **Gambar**.
2. Klik **Upload** dan pilih foto JPG atau PNG.
3. Tunggu satu sampai dua detik sampai hasilnya muncul.



Gambar 7.3 Hasil deteksi, pekerja tanpa rompi dan masker diberi kotak merah, muncul peringatan pelanggaran

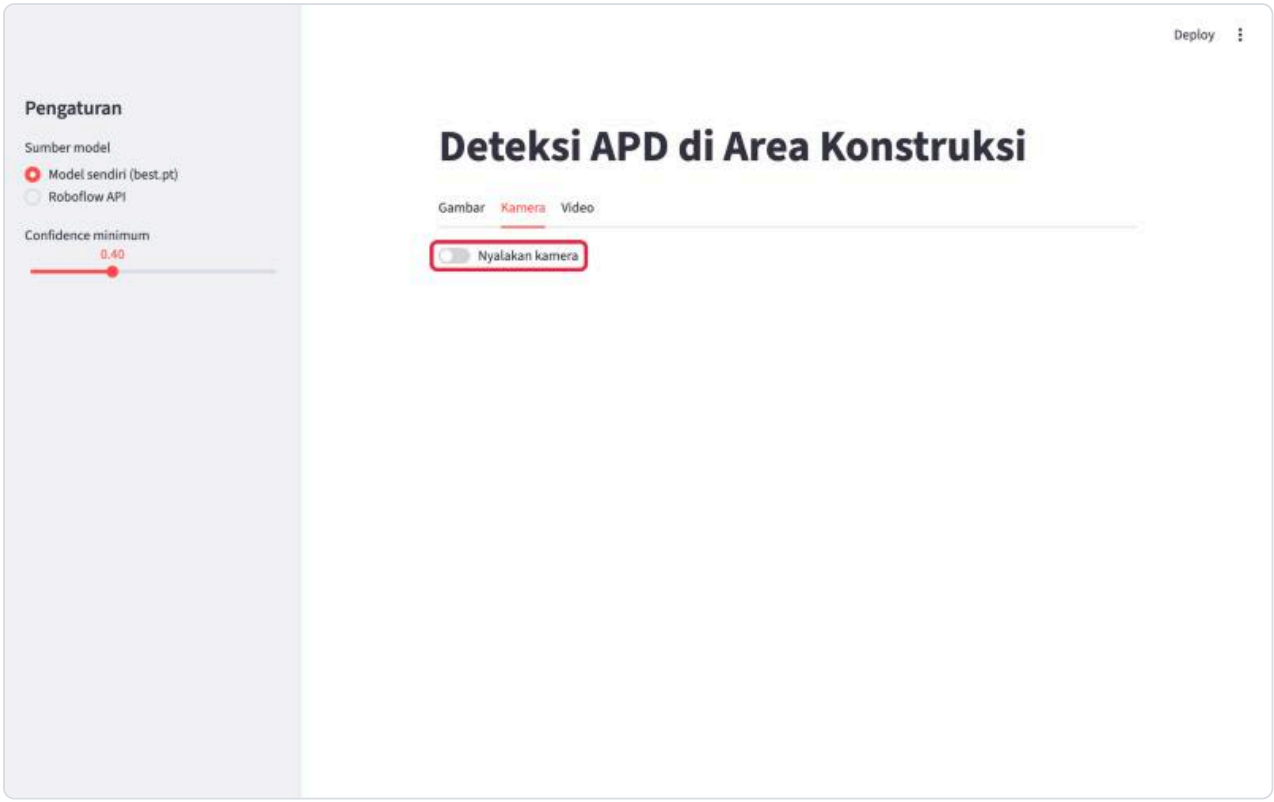
Kotak hijau berarti aman, kotak merah berarti pelanggaran. Angka di sebelah nama kelas adalah tingkat keyakinan model. Gulir ke bawah untuk melihat jumlah setiap objek.



Gambar 7.4 Peringatan pelanggaran dan ringkasan jumlah objek yang terdeteksi di bawah gambar

Langkah 4: Deteksi dari kamera

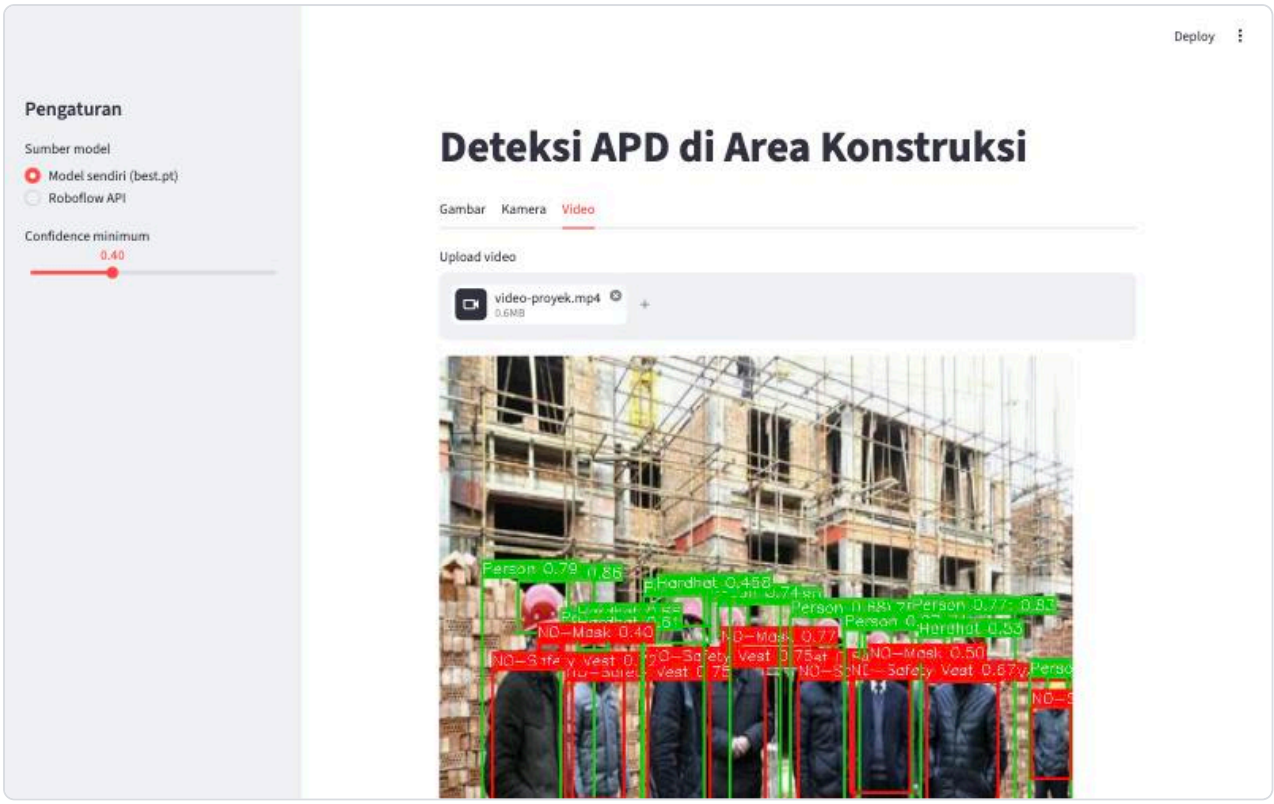
1. Buka tab **Kamera** dan nyalakan tombol **Nyalakan kamera**.
2. Klik **Allow** saat browser meminta izin kamera.
3. Klik **Take Photo**. Hasil deteksi muncul di bawah foto.



Gambar 7.5 Tab Kamera, tombol Nyalakan kamera ditandai

Langkah 5: Deteksi dari video

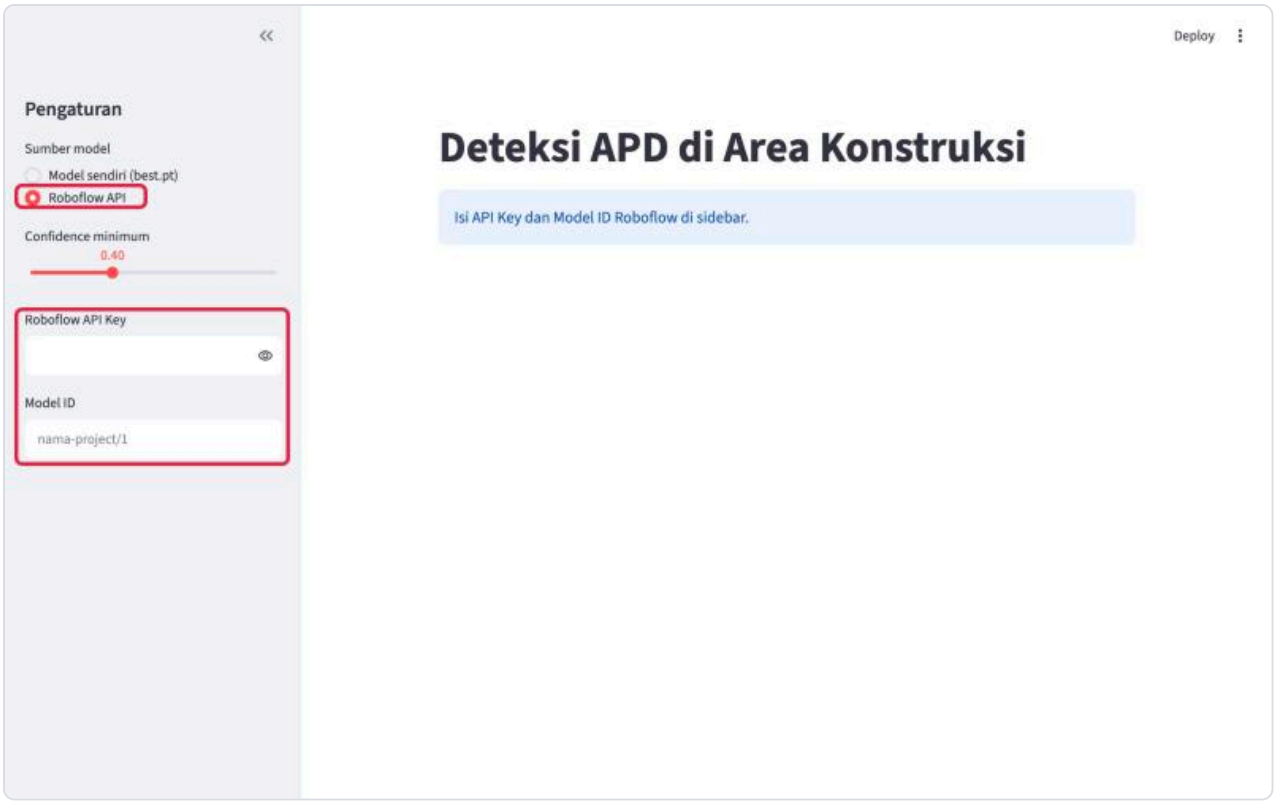
Buka tab **Video** dan upload file MP4. Web memproses video per frame dan menampilkan hasilnya bergantian. Fitur ini hanya bisa dipakai dengan model sendiri.



Gambar 7.6 Tab Video sedang memutar hasil deteksi frame demi frame

Langkah 6: Pakai model Roboflow

1. Di sidebar, pilih **Roboflow API**.
2. Isi **Roboflow API Key** dan **Model ID** yang Anda salin di Bab 5.
3. Kembali ke tab Gambar dan upload foto seperti biasa.



Gambar 7.7 Sidebar dengan pilihan Roboflow API, kolom API Key dan Model ID ditandai

8. Memahami Kode Web Deteksi

Bab ini membedah `app/app.py` dari baris paling atas sampai bawah. Anda tidak perlu menghafalnya. Cukup pahami garis besarnya supaya tahu bagian mana yang perlu diubah kalau ingin menambah fitur.

Istilah Python yang dipakai

Istilah	Contoh di kode	Artinya
Library	`import cv2`	Kumpulan kode siap pakai buatan orang lain
Variabel	`MODEL_PATH = "models/best.pt"`	Nama untuk menyimpan sebuah nilai
Fungsi	`def muat_model():`	Sekumpulan perintah yang diberi nama dan bisa dipanggil berkali-kali
return	`return deteksi`	Nilai yang dikirim balik oleh fungsi ke bagian yang memanggilnya
List	`["NO-Hardhat", "NO-Mask"]`	Daftar berisi beberapa nilai berurutan
Dictionary	`{"Hardhat": 2, "Person": 3}`	Pasangan nama dan nilai
for	`for box in hasil.boxes:`	Mengulang perintah untuk setiap isi daftar
if	`if file:`	Menjalankan perintah hanya kalau syaratnya terpenuhi

Baris yang diawali `#` adalah komentar. Python melewatinya; isinya catatan untuk manusia yang membaca kode.

Bagian 1: Memanggil library

```
import os
import tempfile

import cv2
import numpy as np
import streamlit as st
from PIL import Image
from ultralytics import YOLO
from inference_sdk import InferenceHTTPClient
```

Sebelum dipakai, setiap library harus dipanggil dengan `import`. Ini fungsi masing-masing:

Library	Dipakai untuk
`os`	Mengecek apakah file `models/best.pt` sudah ada
`tempfile`	Menyimpan sementara video yang di-upload supaya bisa dibaca OpenCV
`cv2` (OpenCV)	Membaca video per frame, lalu menggambar kotak dan tulisan di atas gambar
`numpy`	Mengubah gambar menjadi kumpulan angka yang bisa diolah model
`streamlit`	Membuat tampilan web: judul, slider, tombol upload, tab
`PIL` (Pillow)	Membuka file gambar hasil upload atau foto kamera
`ultralytics`	Memuat dan menjalankan model YOLO (`best.pt`)
`inference_sdk`	Mengirim gambar ke server Roboflow untuk Jalur A

Ada tiga bentuk penulisan:

- `import cv2` memanggil seluruh library.
- `from PIL import Image` hanya mengambil satu bagian dari library, yaitu `Image`.
- `import numpy as np` memberi nama pendek. Setelah itu kita cukup menulis `np`, bukan `numpy`.

Bagi komputer, sebuah gambar adalah tabel angka. Foto berukuran 640×640 piksel punya 640 baris dan 640 kolom, dan setiap titik berisi tiga angka 0 sampai 255 untuk warna merah, hijau, dan biru. Itulah sebabnya kita butuh `numpy`: library ini jago mengolah tabel angka sebesar itu.

Bagian 2: Pengaturan awal

```
MODEL_PATH = "models/best.pt"
PELANGGARAN = ["NO-Hardhat", "NO-Mask", "NO-Safety Vest"]
```

Dua variabel ini adalah pengaturan yang paling mungkin Anda ubah. `MODEL_PATH` adalah lokasi file model. `PELANGGARAN` adalah daftar kelas yang dianggap pelanggaran dan diberi kotak merah. Nama variabel ditulis huruf besar semua sebagai tanda bahwa nilainya tidak berubah selama program berjalan.

Bagian 3: Memuat model

```
@st.cache_resource
def muat_model():
    return YOLO(MODEL_PATH)
```

Fungsi ini membaca file `best.pt` dan mengembalikan model yang siap dipakai.

Baris `@st.cache_resource` penting. Streamlit menjalankan ulang seluruh file dari atas setiap kali Anda menggeser slider atau meng-upload gambar. Tanpa baris ini, model akan dibaca ulang dari disk setiap kali, dan web terasa lambat. Dengan baris ini, model cukup dibaca sekali lalu disimpan di memori.

Bagian 4: Deteksi dengan model sendiri

```
def deteksi_lokal(gambar, conf):
    hasil = muat_model()(gambar, conf=conf, verbose=False)[0]
    deteksi = []
    for box in hasil.boxes:
        x1, y1, x2, y2 = box.xyxy[0].tolist()
        nama = hasil.names[int(box.cls)]
        deteksi.append((nama, float(box.conf), x1, y1, x2, y2))
    return deteksi
```

Fungsi ini menerima gambar, menjalankan model, lalu merapikan hasilnya menjadi daftar yang mudah dipakai. Baris demi baris:

1. `def deteksi_lokal(gambar, conf):` membuat fungsi dengan dua masukan: `gambar` dan `conf`, yaitu batas keyakinan dari slider di sidebar.
2. `muat_model()(gambar, ...)` menjalankan model pada gambar. `conf=conf` membuang kotak yang keyakinannya di bawah batas, dan `verbose=False` membuat terminal tidak penuh

tulisan. Model selalu mengembalikan daftar hasil, satu untuk setiap gambar. Karena kita hanya mengirim satu gambar, `[0]` mengambil hasil yang pertama.

3. `deteksi = []` menyiapkan daftar kosong untuk menampung hasil.
4. `for box in hasil.boxes:` mengulang langkah 5 sampai 7 untuk setiap kotak yang ditemukan.
5. `box.xyxy[0].tolist()` mengambil posisi kotak: pojok kiri atas (`x1`, `y1`) dan pojok kanan bawah (`x2`, `y2`), dalam satuan piksel.
6. `box.cls` berisi nomor kelas, misalnya 7. `hasil.names[7]` mengubah nomor itu menjadi nama, yaitu "Safety Vest".
7. `deteksi.append(...)` menambahkan satu baris ke daftar: nama kelas, tingkat keyakinan (0 sampai 1), dan empat angka posisi.
8. `return deteksi` mengirim daftar itu kembali ke bagian yang memanggil fungsi ini.

Contoh isi `deteksi` untuk foto dua pekerja di Bab 7:

```
[("Hardhat", 0.92, 140.0, 55.0, 210.0, 120.0),
 ("NO-Safety Vest", 0.83, 300.0, 130.0, 420.0, 400.0)]
```

Satu baris mewakili satu kotak. Format ini sengaja dibuat sama dengan hasil fungsi Roboflow di bagian berikutnya, supaya fungsi penggambar kotak bisa dipakai untuk keduanya.

Bagian 5: Deteksi dengan Roboflow

```
def deteksi_roboflow(gambar, conf, api_key, model_id):
    client = InferenceHTTPClient(api_url="https://serverless.roboflow.com",
api_key=api_key)
    hasil = client.infer(Image.fromarray(gambar), model_id=model_id)
    deteksi = []
    for p in hasil["predictions"]:
        if p["confidence"] < conf:
            continue
        x1, y1 = p["x"] - p["width"] / 2, p["y"] - p["height"] / 2
        x2, y2 = p["x"] + p["width"] / 2, p["y"] + p["height"] / 2
        deteksi.append((p["class"], p["confidence"], x1, y1, x2, y2))
    return deteksi
```

Tugasnya sama dengan Bagian 4, tetapi gambar dikirim ke server Roboflow lewat internet.

1. `InferenceHTTPClient(...)` membuat koneksi ke server Roboflow memakai API Key Anda.
2. `client.infer(...)` mengirim gambar dan menyebutkan model mana yang dipakai (Model ID). Server membalas dengan dictionary yang berisi daftar `predictions`.

3. `if p["confidence"] < conf: continue` melewati kotak yang keyakinannya terlalu rendah. `continue` artinya lompat ke kotak berikutnya.
4. Roboflow memberi posisi kotak dalam bentuk titik tengah (`x` , `y`) serta lebar dan tinggi. Dua baris rumus mengubahnya menjadi pojok kiri atas dan kanan bawah, sama seperti format YOLO. Contoh: titik tengah x = 200 dan lebar = 100, berarti sisi kiri ada di 200 - 50 = 150 dan sisi kanan di 200 + 50 = 250.

Bagian 6: Menggambar kotak

```
def gambar_kotak(gambar, deteksi):
    gambar = gambar.copy()
    for nama, skor, x1, y1, x2, y2 in deteksi:
        warna = (255, 0, 0) if nama in PELANGGARAN else (0, 200, 0)
        p1, p2 = (int(x1), int(y1)), (int(x2), int(y2))
        teks = f"{nama} {skor:.2f}"
        (w, h), _ = cv2.getTextSize(teks, cv2.FONT_HERSHEY_SIMPLEX, 0.5, 1)
        cv2.rectangle(gambar, p1, p2, warna, 2)
        cv2.rectangle(gambar, (p1[0], p1[1] - h - 6), (p1[0] + w, p1[1]), warna,
-1)
        cv2.putText(gambar, teks, (p1[0], p1[1] - 4),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 1)
    return gambar
```

- `gambar.copy()` membuat salinan, supaya gambar aslinya tidak ikut tercoret.
- `for nama, skor, x1, y1, x2, y2 in deteksi:` membongkar setiap baris hasil deteksi menjadi enam variabel sekaligus.
- Warna ditulis sebagai tiga angka (merah, hijau, biru). `(255, 0, 0)` adalah merah, `(0, 200, 0)` adalah hijau. Kotak menjadi merah kalau nama kelasnya ada di daftar `PELANGGARAN` .
- `f"{nama} {skor:.2f}"` menyusun tulisan label, misalnya `Hardhat 0.92` . Bagian `:.2f` membulatkan angka menjadi dua digit di belakang koma.
- `cv2.getTextSize` mengukur lebar dan tinggi tulisan, supaya latar label bisa dibuat pas.
- `cv2.rectangle` dipanggil dua kali. Yang pertama menggambar garis kotak setebal 2 piksel. Yang kedua, dengan ketebalan `-1` , menggambar kotak penuh berwarna sebagai latar tulisan.
- `cv2.putText` menulis label berwarna putih di atas latar tadi.

Bagian 7: Ringkasan dan peringatan

```
def tampilkan_ringkasan(deteksi):
    jumlah = {}
    for d in deteksi:
        jumlah[d[0]] = jumlah.get(d[0], 0) + 1

    pelanggaran = {k: v for k, v in jumlah.items() if k in PELANGGARAN}
    if pelanggaran:
        for nama, n in pelanggaran.items():
            st.error(f"{n} pelanggaran: {nama}")
    else:
        st.success("Tidak ada pelanggaran APD")
```

- `jumlah = {}` membuat dictionary kosong untuk menghitung objek per kelas.
- `d[0]` adalah nama kelas (isi pertama setiap baris deteksi). `jumlah.get(d[0], 0) + 1` menambah hitungan satu. Kalau kelas itu belum pernah muncul, hitungan dimulai dari 0.
- Baris `pelanggaran = ...` menyaring dictionary, sehingga yang tersisa hanya kelas pelanggaran.
- `st.error` menampilkan kotak peringatan merah di web. `st.success` menampilkan kotak hijau.

Setelah itu fungsi menulis daftar semua objek yang terdeteksi dengan `st.write`.

Bagian 8: Tampilan web

```
st.set_page_config(page_title="Deteksi APD")
st.title("Deteksi APD di Area Konstruksi")

st.sidebar.header("Pengaturan")
sumber = st.sidebar.radio("Sumber model", ["Model sendiri (best.pt)", "Roboflow
API"])
conf = st.sidebar.slider("Confidence minimum", 0.1, 1.0, 0.4)
```

Setiap perintah `st.` langsung menjadi elemen di halaman web. `st.title` menjadi judul, `st.sidebar.radio` menjadi pilihan di sidebar, dan `st.sidebar.slider` menjadi slider dengan nilai terkecil 0.1, terbesar 1.0, dan nilai awal 0.4. Pilihan pengguna langsung tersimpan di variabel `sumber` dan `conf`.

Kode berikutnya memeriksa kesiapan. Kalau memilih Roboflow tetapi API Key belum diisi, atau file `best.pt` belum ada, web menampilkan pesan lalu berhenti dengan `st.stop()`.

```
tab_gambar, tab_kamera, tab_video = st.tabs(["Gambar", "Kamera", "Video"])

with tab_gambar:
    file = st.file_uploader("Upload gambar", type=["jpg", "jpeg", "png"])
    if file:
        gambar = np.array(Image.open(file).convert("RGB"))
        hasil = deteksi(gambar)
        st.image(gambar_kotak(gambar, hasil))
        tampilkan_ringkasan(hasil)
```

- `st.tabs` membuat tiga tab. Semua kode yang menjorok di bawah `with tab_gambar:` hanya tampil di tab Gambar.
- `if file:` berarti bagian di bawahnya baru berjalan setelah pengguna meng-upload gambar.
- `np.array(Image.open(file).convert("RGB"))` membuka file, memastikan warnanya RGB, lalu mengubahnya menjadi tabel angka.
- Tiga baris terakhir memanggil fungsi-fungsi dari Bagian 4 sampai 7 secara berurutan.

Tab Video bekerja dengan cara yang mirip, hanya saja diulang untuk setiap frame. Video disimpan dulu ke file sementara dengan `tempfile`, lalu dibaca satu per satu dengan `cv2.VideoCapture`. OpenCV membaca warna dengan urutan biru, hijau, merah, sehingga `cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)` dipakai untuk membaliknya.

Alur lengkap dari foto sampai peringatan

1. Pengguna meng-upload foto di tab Gambar.
2. Foto diubah menjadi tabel angka dengan `np.array`.
3. Fungsi `deteksi()` memilih jalur sesuai sidebar: `deteksi_lokal` atau `deteksi_roboflow`.
4. Hasilnya berupa daftar kotak: nama kelas, keyakinan, dan posisi.
5. `gambar_kotak` menggambar kotak merah atau hijau di atas foto, lalu `st.image` menampilkannya.
6. `tampilkan_ringkasan` menghitung objek dan menampilkan peringatan pelanggaran.

Latihan

□ Ganti judul web di baris `st.title(...)` dengan nama perusahaan Anda, lalu simpan file dan muat ulang browser.

□ Ubah nilai awal slider dari `0.4` menjadi `0.6`, lalu bandingkan jumlah kotak yang muncul.

□ Tambahkan `"Person"` ke daftar `PELANGGARAN`, lalu lihat apa yang berubah pada warna kotak. Setelah itu kembalikan seperti semula.

□ Ganti warna hijau `(0, 200, 0)` menjadi biru `(0, 0, 255)`.

9. Menyimpan Proyek ke GitHub

Git mencatat setiap perubahan kode di laptop Anda. GitHub menyimpan catatan itu secara online, sehingga proyek tetap aman walaupun laptop rusak, dan teman satu tim bisa ikut mengerjakan.

Istilah yang akan sering muncul

Istilah	Maksudnya
Repository (repo)	Folder proyek yang dicatat Git
Commit	Titik simpan berisi perubahan dan pesan singkat
Push	Mengirim commit dari laptop ke GitHub
Pull	Mengambil perubahan terbaru dari GitHub ke laptop
Clone	Mengunduh repo dari GitHub ke laptop
Branch	Cabang kerja terpisah untuk mencoba fitur baru
Pull request (PR)	Permintaan menggabungkan branch ke versi utama

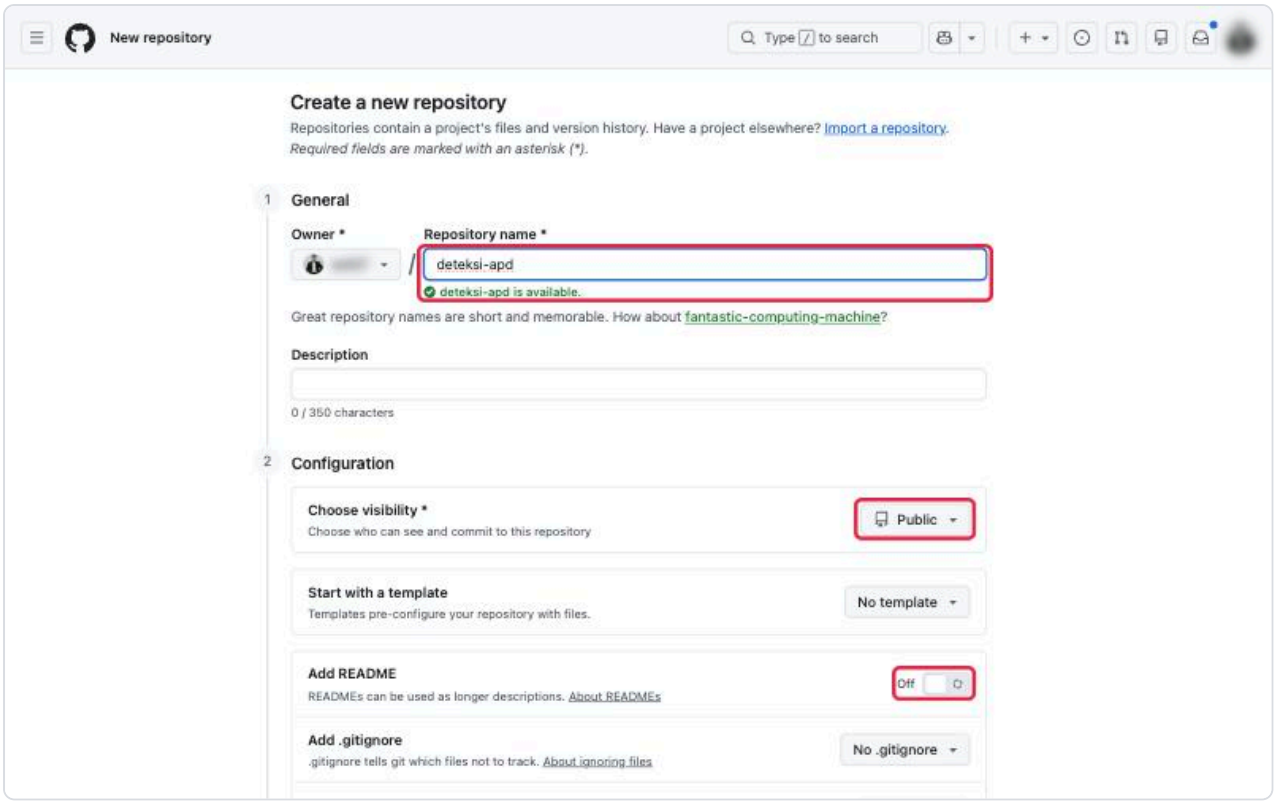
Langkah 1: Isi nama dan email (sekali saja)

```
git config --global user.name "Nama Anda"
git config --global user.email "email-github@anda.com"
```

Pakai email yang sama dengan akun GitHub Anda.

Langkah 2: Buat repository di GitHub

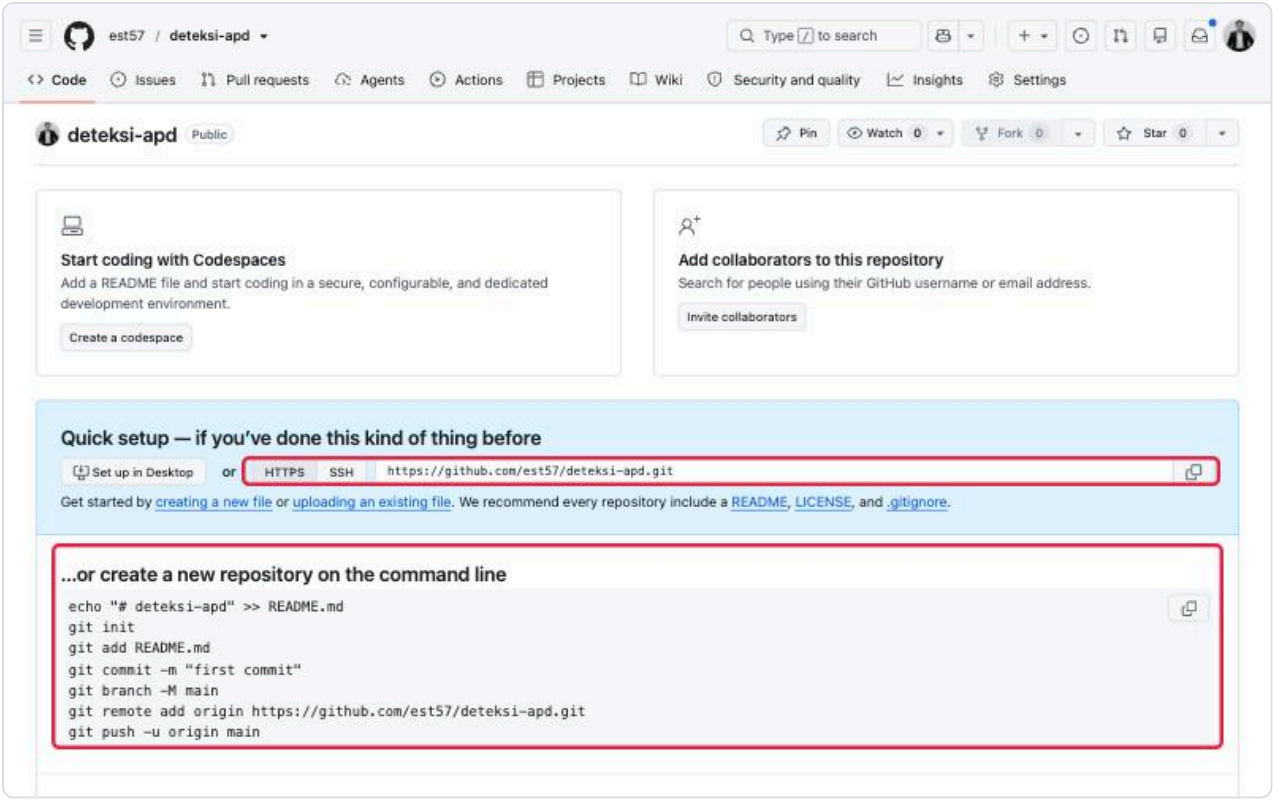
1. Login ke github.com. Klik tombol **+** di kanan atas, lalu **New repository**.
2. Isi **Repository name**, misalnya `deteksi-apd`. Tanda centang hijau berarti nama itu belum Anda pakai.
3. Biarkan visibility di **Public**.
4. Biarkan **Add README** di posisi Off, karena proyek kita sudah punya README.



Gambar 9.1 Form New repository, kolom nama, pilihan Public, dan Add README ditandai

5. Gulir ke bawah dan klik **Create repository**.

GitHub lalu membuka halaman Quick setup. Alamat di kotak HTTPS adalah alamat repo Anda, dan kotak di bawahnya berisi perintah yang akan kita pakai di langkah 3.



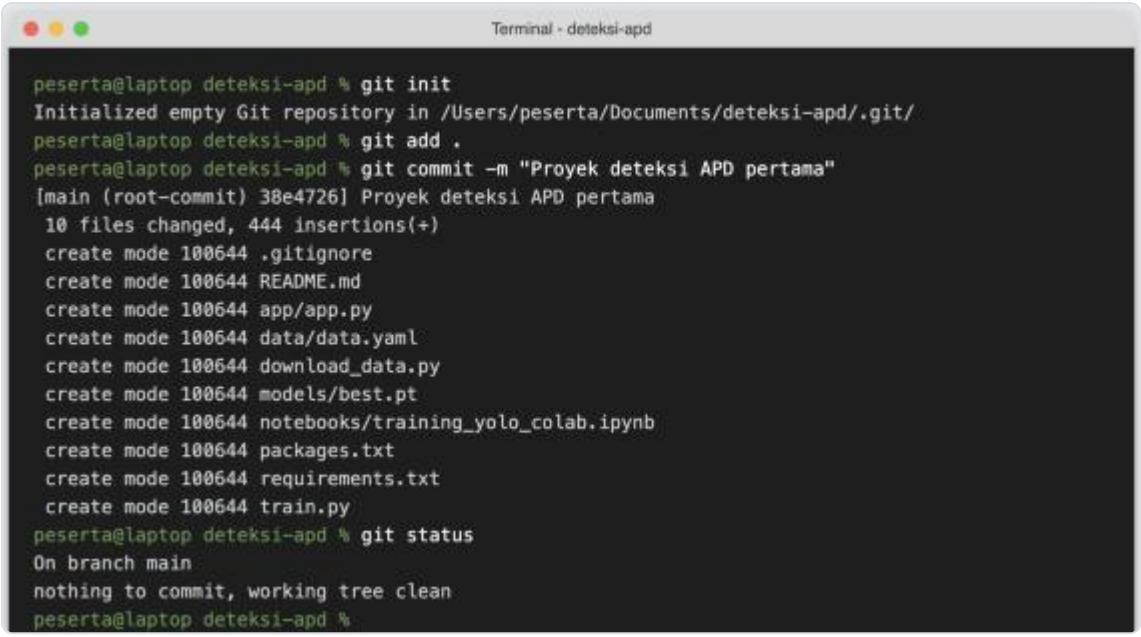
Gambar 9.2 Halaman Quick setup, alamat HTTPS repo dan daftar perintah ditandai

Langkah 3: Upload proyek lewat terminal

Jalankan perintah di bawah dari terminal VS Code, di dalam folder proyek. Ganti `USERNAME` dengan username GitHub Anda.

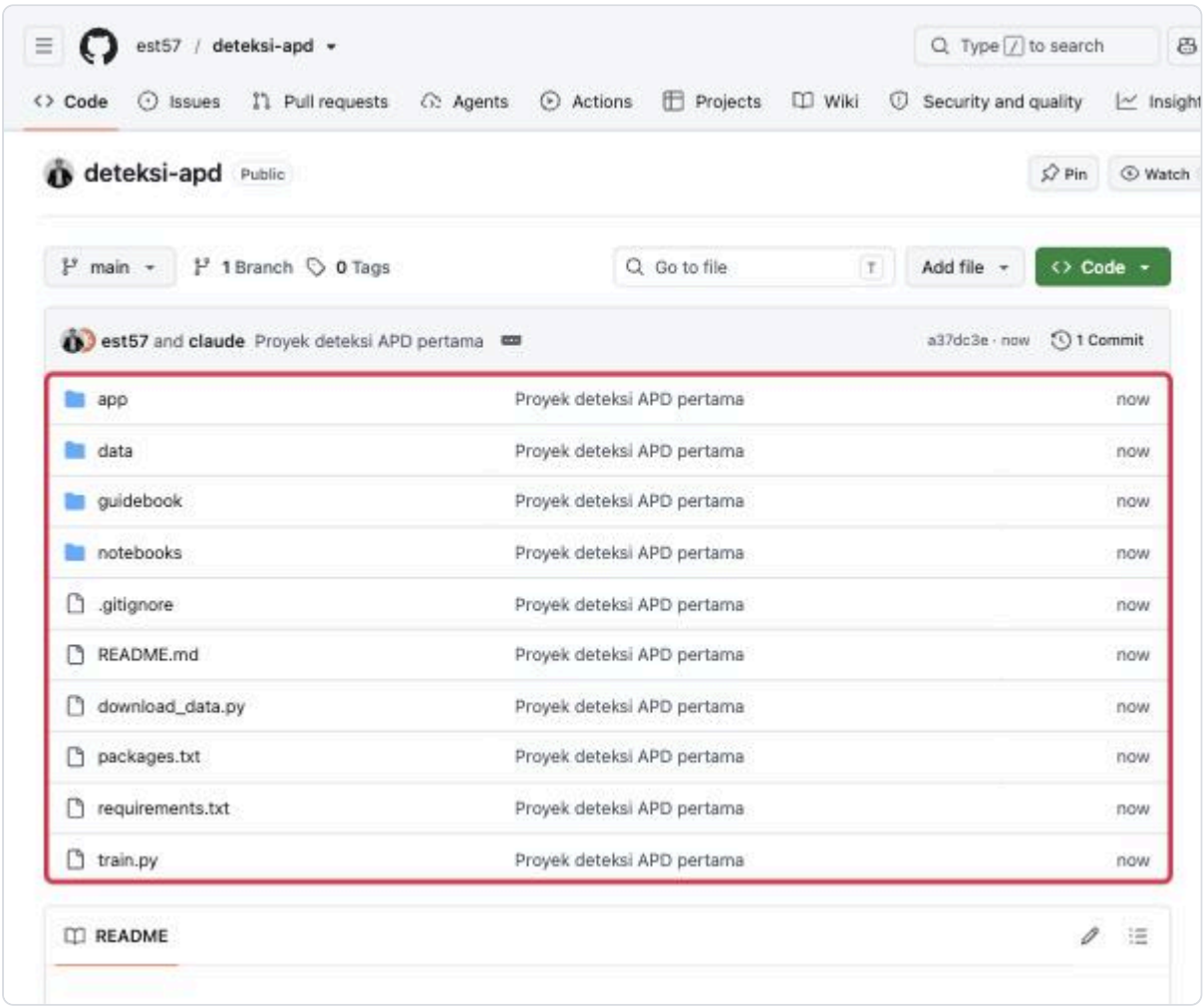
```
git init
git add .
git commit -m "Proyek deteksi APD pertama"
git branch -M main
git remote add origin https://github.com/USERNAME/deteksi-apd.git
git push -u origin main
```

Setelah `git commit`, terminal menampilkan daftar file yang tercatat seperti gambar berikut.



Gambar 9.3 Terminal setelah git init, git add, git commit, dan git status

Saat `git push` pertama kali, browser membuka halaman login GitHub. Login, klik **Authorize**, lalu muat ulang halaman repo. File proyek Anda sekarang sudah ada di sana.



Gambar 9.4 Halaman repo deteksi-apd di GitHub setelah push, daftar file proyek ditandai

Cara lain: lewat tombol di VS Code

Kalau belum nyaman dengan terminal, VS Code bisa mengerjakan langkah 2 dan 3 sekaligus.

1. Klik ikon **Source Control** (gambar cabang) di sisi kiri VS Code.
2. Klik **Publish to GitHub**.
3. Login GitHub kalau diminta, lalu pilih **Publish to GitHub public repository**.

File yang sengaja tidak di-upload

File `.gitignore` berisi daftar folder yang dilewati Git: `.venv`, `data/css-data`, dan `runs`. Ukurannya besar dan bisa dibuat ulang kapan saja. Model `models/best.pt` (sekitar 6 MB) tetap ikut supaya orang lain bisa langsung menjalankan web.

Alur kerja sehari-hari

Setiap kali selesai mengubah kode:

```
git status
git add .
git commit -m "Ganti judul web"
git push
```

`git status` menampilkan file mana saja yang berubah. Tulis pesan commit yang menjelaskan isi perubahan, misalnya "Tambah deteksi video". Pesan seperti "update" akan membingungkan Anda sendiri sebulan kemudian.

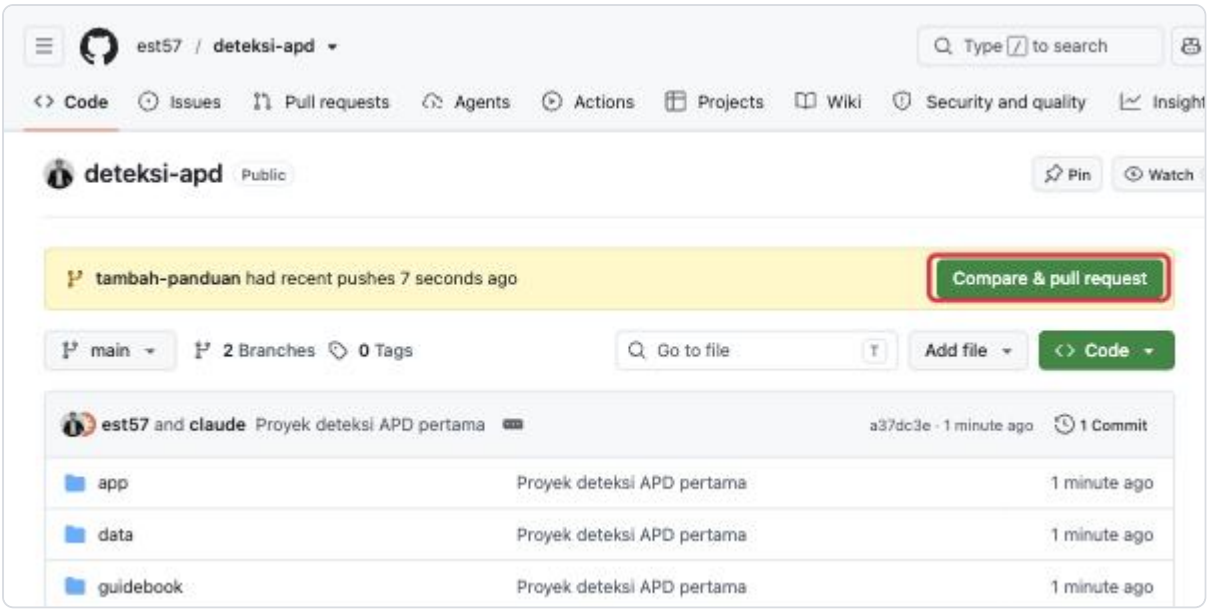
Branch dan pull request

Buat branch baru setiap kali mencoba fitur, supaya versi utama di `main` tetap bisa dipakai.

```
git checkout -b judul-baru
# ubah kode di VS Code
git add .
git commit -m "Ganti judul web"
git push -u origin judul-baru
```

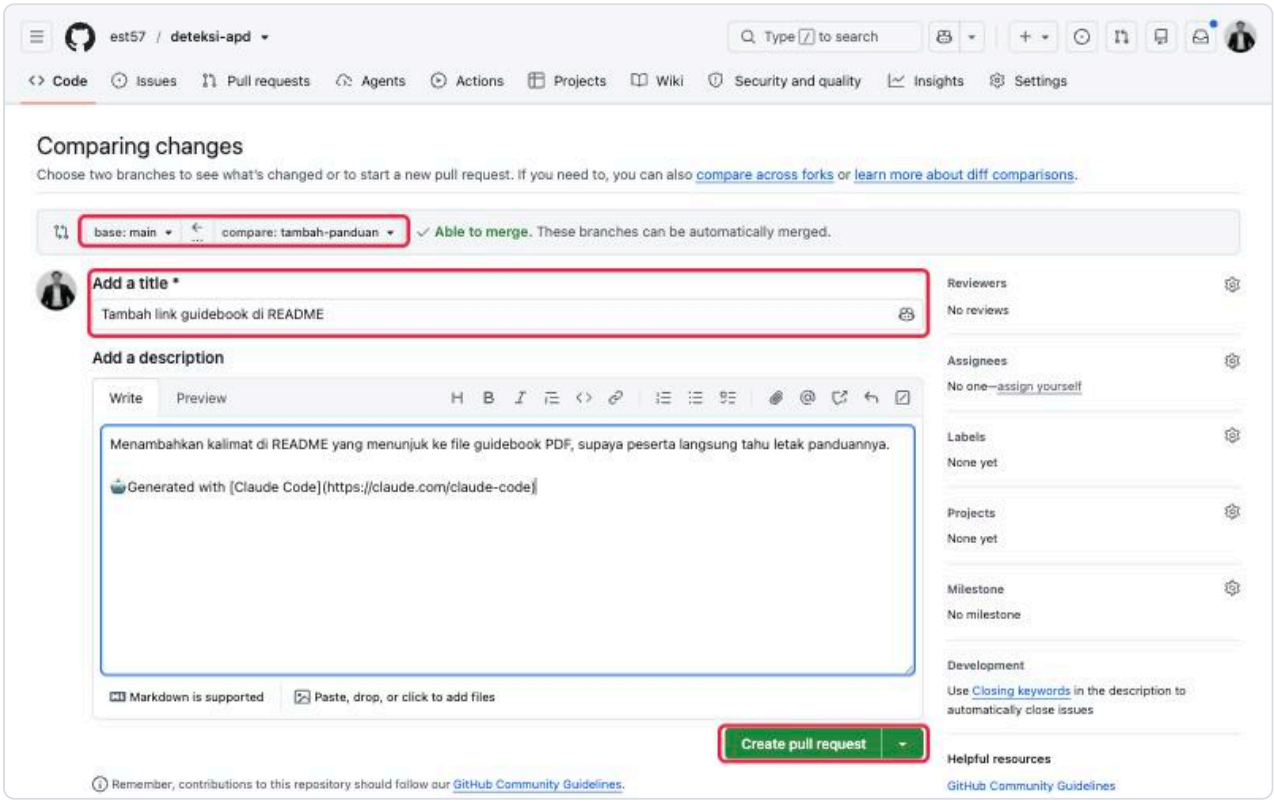
Lanjutkan di GitHub:

1. Buka halaman repo. Klik **Compare & pull request** di banner kuning.



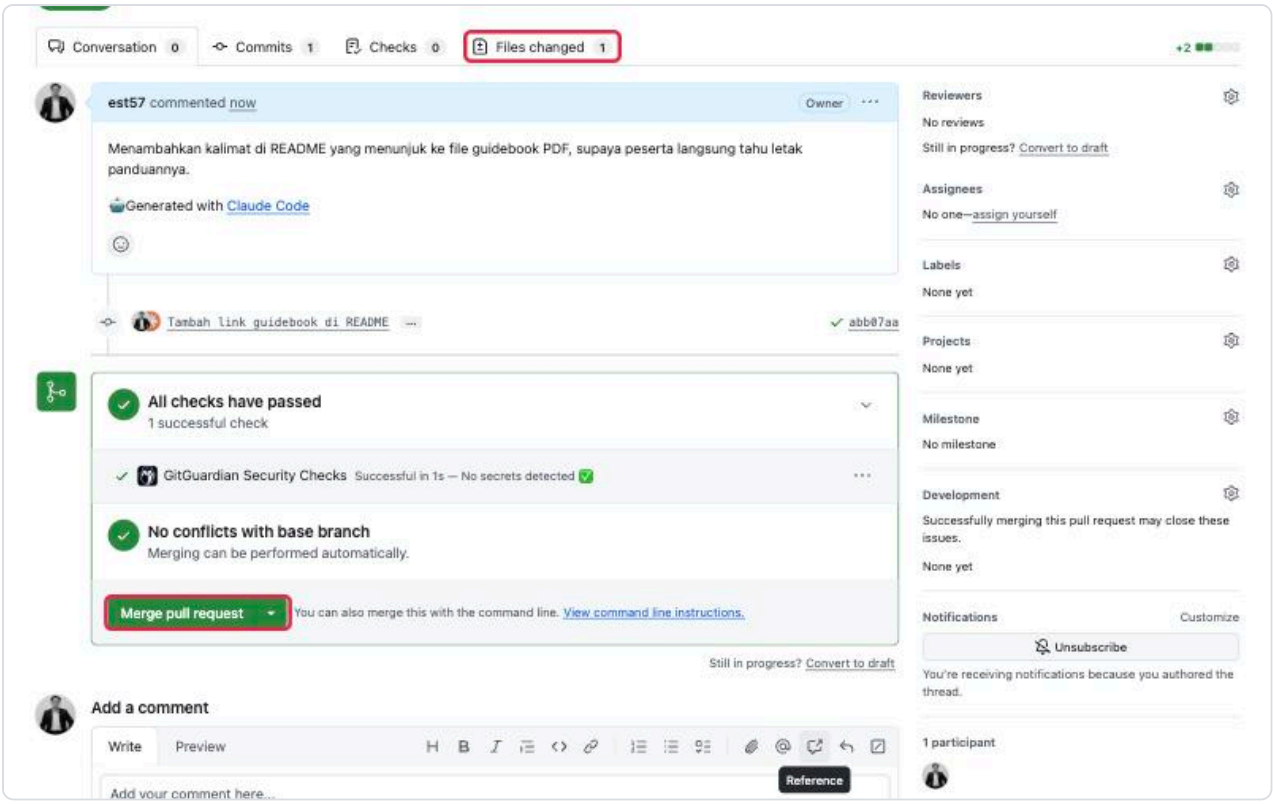
Gambar 9.5 Banner kuning di halaman repo, tombol Compare & pull request ditandai

2. Pastikan tulisan di atas form adalah **base: main** dan **compare: nama-branch Anda**. Isi judul dan penjelasan singkat, lalu klik **Create pull request**.



Gambar 9.6 Form Open a pull request, pilihan branch, judul, dan tombol Create pull request ditandai

3. Di halaman pull request, buka tab **Files changed** untuk memeriksa perubahan.
4. Kalau sudah yakin, klik **Merge pull request**, lalu **Confirm merge**.



Gambar 9.7 Halaman pull request, tab Files changed dan tombol Merge pull request ditandai

Kembali ke laptop dan ambil versi terbaru:

```
git checkout main
git pull
```

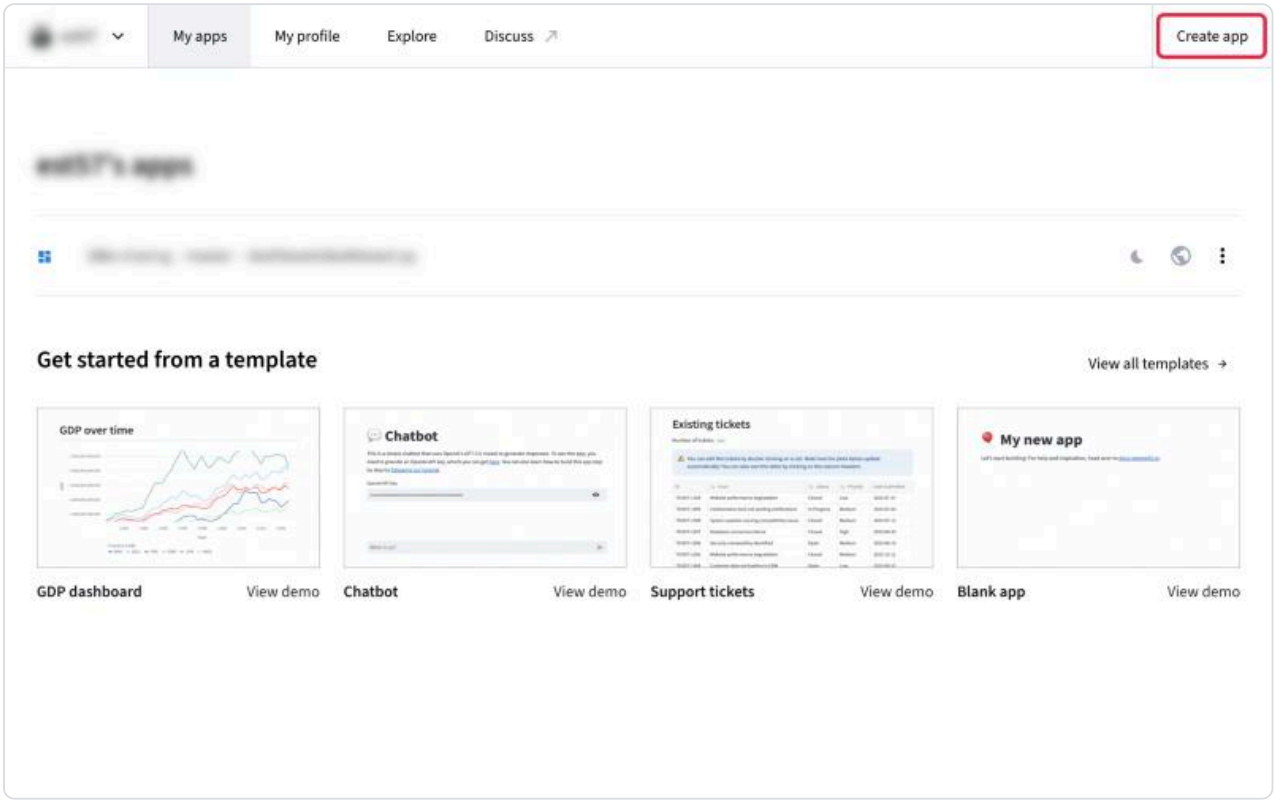
Mengunduh proyek orang lain

```
git clone https://github.com/USERNAME/deteksi-apd.git
```

10. Bonus: Membuat Web Bisa Dibuka dari Mana Saja

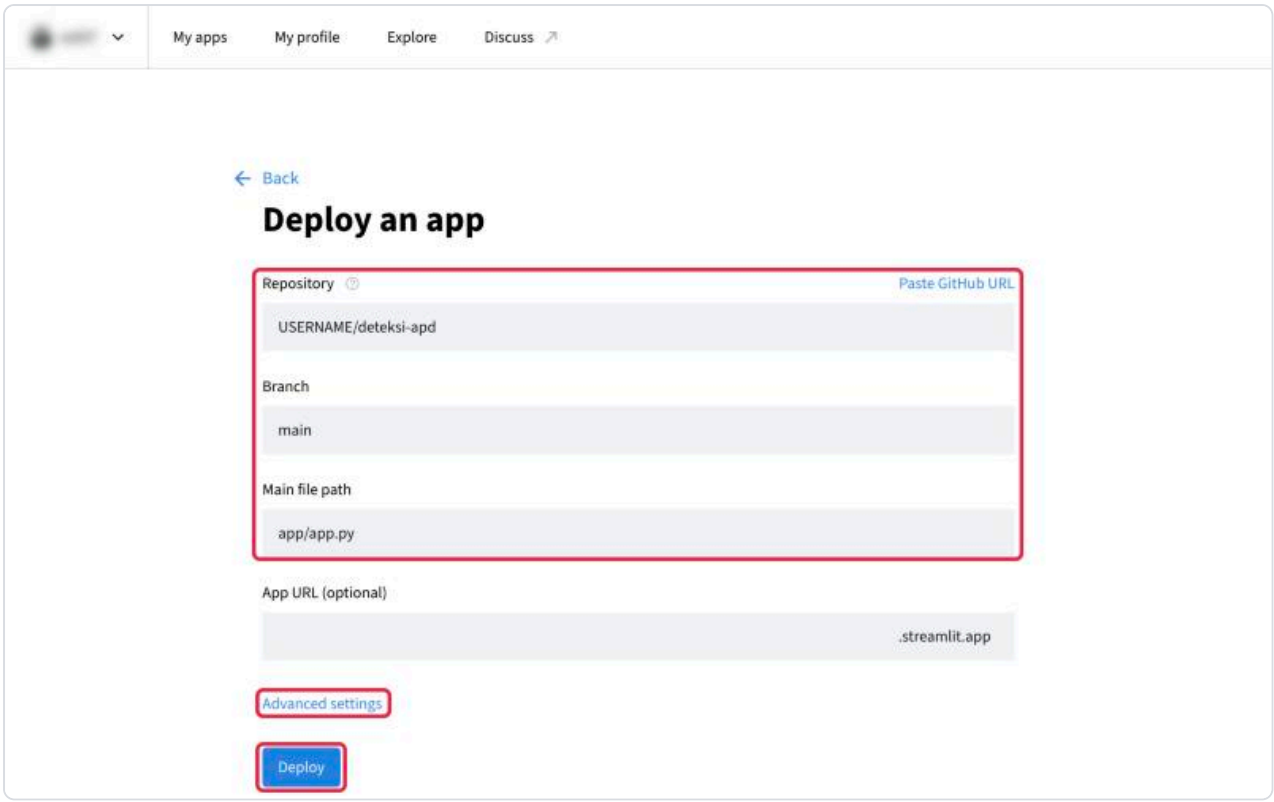
Streamlit Community Cloud menjalankan web Anda di server mereka secara gratis. Setelah itu, web bisa dibuka dari HP atau dikirim ke atasan lewat link.

1. Buka share.streamlit.io dan login dengan akun GitHub.
2. Klik **Create app** di kanan atas.



Gambar 10.1 Halaman My apps di Streamlit Community Cloud, tombol Create app ditandai

3. Pilih **Deploy a public app from GitHub**, lalu klik **Deploy now**.
4. Isi formulirnya: - **Repository**: `USERNAME/deteksi-apd` - **Branch**: `main` - **Main file path**: `app/app.py`
5. Klik **Advanced settings** dan pilih Python **3.12**.
6. Klik **Deploy**, lalu tunggu 3 sampai 5 menit.



Gambar 10.2 Form Deploy an app, kolom Repository, Branch, Main file path, Advanced settings, dan tombol Deploy ditandai

Anda akan mendapat alamat seperti `https://deteksi-apd.streamlit.app`. Setiap kali Anda menjalankan `git push`, web online ikut diperbarui.

Jangan menghapus file `packages.txt`. Server Streamlit membutuhkannya untuk menjalankan OpenCV.

11. Kalau Ada Masalah

Masalah	Yang perlu dilakukan
`python` tidak dikenali (Windows)	Install ulang Python dan centang Add python.exe to PATH
`python` tidak dikenali (Mac)	Pakai `python3` dan `pip3`
Windows menolak menjalankan `.venv\Scripts\activate`	Buka PowerShell, jalankan `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, lalu coba lagi
Muncul `ModuleNotFoundError`	Virtual environment belum aktif. Aktifkan, lalu jalankan `pip install -r requirements.txt`
Web menulis "File models/best.pt belum ada"	Salin `best.pt` hasil training ke folder `models/`
`streamlit run` gagal menemukan file	Pastikan terminal berada di folder utama proyek, bukan di dalam `app/`
Training di Colab sangat lambat	Cek **Runtime > Change runtime type**, pilih T4 GPU, lalu jalankan ulang dari sel pertama
Colab terputus di tengah training	Jangan tinggalkan tab Colab terlalu lama, lalu jalankan ulang sel training
Roboflow API error 401 atau 403	API Key salah. Salin ulang Private API Key dari Settings
Roboflow API error 404	Model ID salah. Formatnya `nama-project/nomor-versi`
Kamera tidak muncul	Klik ikon gembok di address bar browser dan izinkan kamera
`git push` ditolak	Jalankan `git pull`, lalu ulangi `git push`
Kelas di Roboflow bernama angka 0 sampai 9	data.yaml tidak ikut ter-upload. Buka Classes & Tags, centang kelas angka, klik Rename, lalu ganti: 0 Hardhat, 1 Mask, 2 NO-Hardhat, 3 NO-Mask, 4 NO-Safety Vest, 5 Person, 6 Safety Cone, 7 Safety Vest, 8 machinery, 9 vehicle

Daftar istilah

Istilah	Arti
APD	Alat Pelindung Diri, seperti helm, rompi, dan masker
Object detection	Teknik untuk menemukan objek beserta posisinya di gambar

Istilah	Arti
YOLO	Singkatan dari You Only Look Once, model deteksi objek yang cepat
Dataset	Kumpulan gambar beserta labelnya
Epoch	Satu putaran model membaca seluruh gambar training
Confidence	Tingkat keyakinan model terhadap satu deteksi, dari 0 sampai 1
mAP50	Nilai akurasi model deteksi objek, dari 0 sampai 1
GPU	Prosesor grafis yang mempercepat training
API Key	Kunci rahasia untuk mengakses layanan online seperti Roboflow
Virtual environment	Folder terpisah untuk library satu proyek